

Banco Bolivariano

Octubre
2021

Capacitación Interna BB

Estándares y mejores prácticas de programación en Base de Datos



Calidad de Software

¿QUE ES LA CALIDAD DEL SOFTWARE?

“La calidad del *software* es el conjunto de cualidades que lo caracterizan y que determinan su utilidad y existencia. La calidad es sinónimo de:

- Eficiencia,
- Flexibilidad,
- Corrección,
- Confiabilidad,
- Mantenibilidad,
- Portabilidad,
- Usabilidad,
- Seguridad e Integridad.

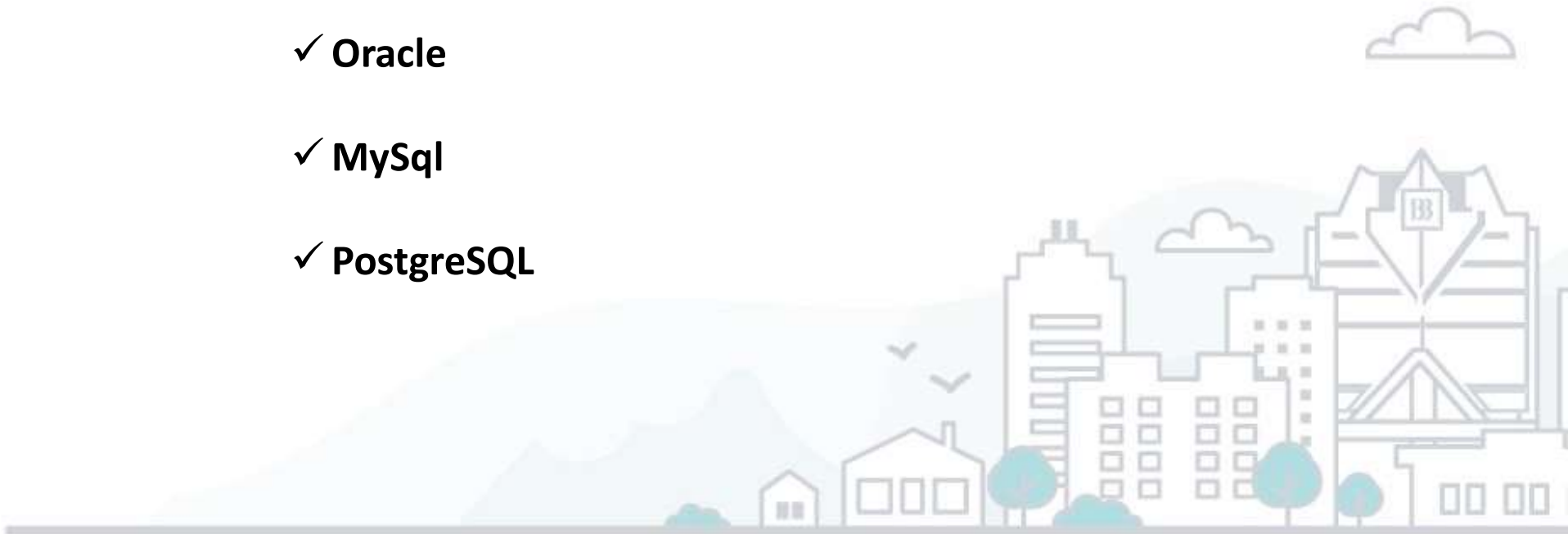
Lograr el éxito en la producción de *software* es hacerlo con calidad y demostrar su buena calidad. ”

Desarrollar software cumpliendo los estándares de calidad, seguridad y de buen rendimiento es un ARTE. - MARCELO DURÁN NAVARRETE



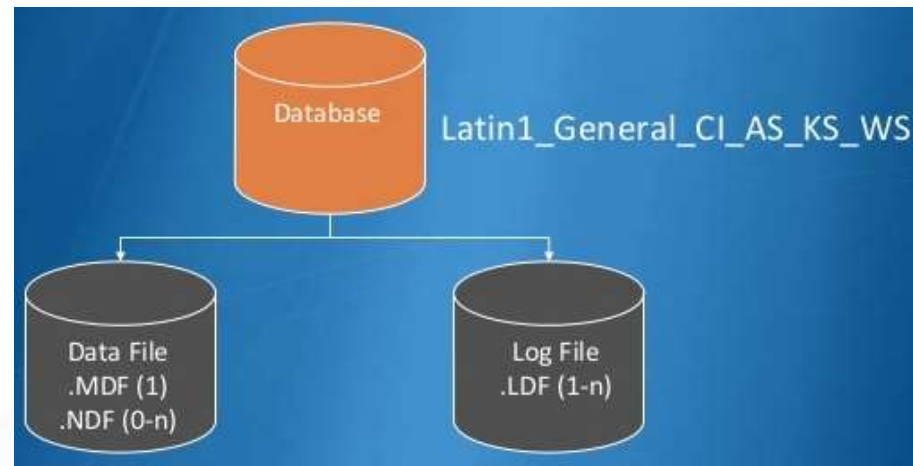
Estándares de programación de base de datos

- ✓ Sybase
- ✓ SqlServer
- ✓ Oracle
- ✓ MySql
- ✓ PostgreSQL



Base de Datos

- ✓ En Sybase y Sql Server la base de datos está conformada por dos áreas: de Datos y de Log.
- ✓ El área de log se estima entre un 10 al 25% del tamaño de el área de datos.
- ✓ En el log de transacciones se almacena los comandos insert/update/delete que son limpiados cuando las transacciones realizan el commit, guardar en disco.



Nombre de la base de datos

- ✓ El nombre de la Base de Datos debe tener una longitud máxima de 25 caracteres y en singular.
- ✓ Si la información almacenada en la base de datos es alimentada por aplicaciones COBIS debe empezar con el prefijo cob_.
- ✓ Si pertenece a aplicaciones no Cobis debe anteponérsele el prefijo db_.
- ✓ Si la base de datos almacena información histórica debe finalizar con _his.

Ejemplos:

cob_cuenta
cob_cuenta_his
db_sat_his
db_externa

- *Cuando sean aplicaciones adquiridas a proveedores externos (como Facturación Electrónica, Evolution, etc) y sean un paquete cerrado, es necesario incluir en la negociación un anexo de la parte técnica donde entreguen la documentación relacionada a: estándar de la base de datos, diccionario de datos, modelo entidad relación, procesos de depuración, permisos especiales – seguridad, según aplique.*

Nemónico de la aplicación

- ✓ El nemónico de la aplicación será las mismas que están establecidas en la herramienta Rational ClearQuest.

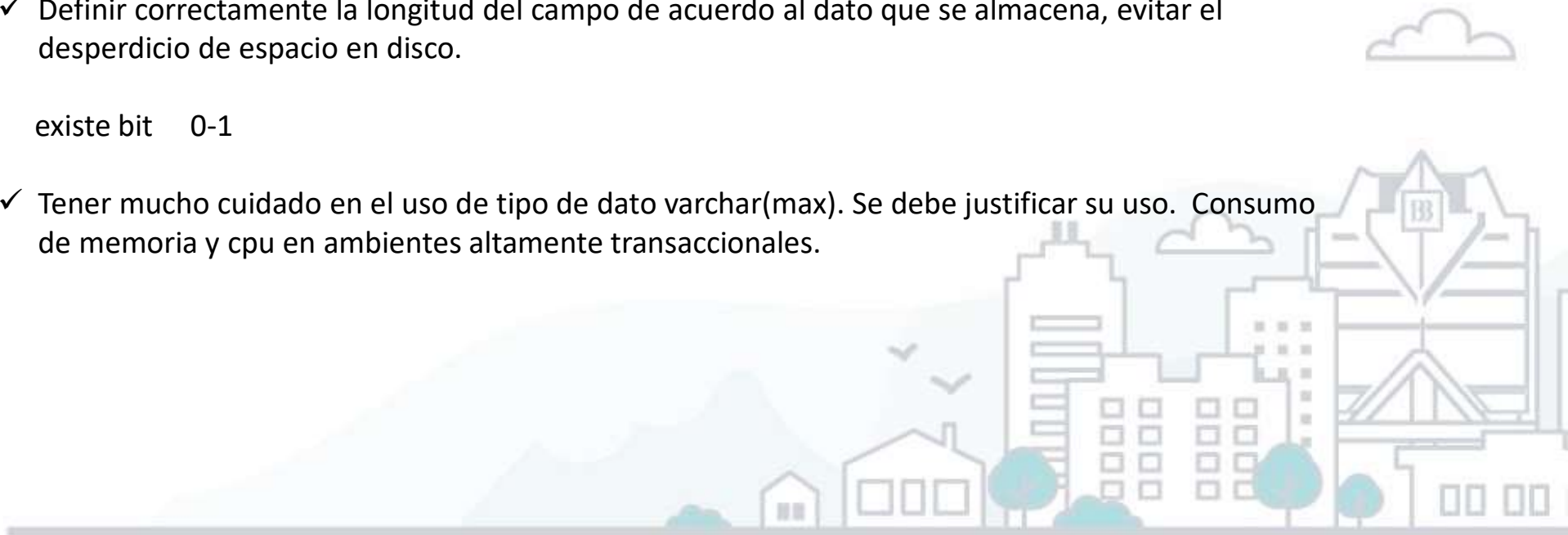
Nemonico	Nombre
TBC	Tarjeta Bancard
PBMO	BBP-24Movil
BMO	24Movil
CHBOT	CHATBOT – Canal de mensajería asistida por Robot
GIR	Grabador Interactivo de Genesys
VSD	VERSADIAL
CAL	Call Center
INTRA	Intranet
Octadocs	Administrador de Procedimientos
REC	Reclamos
IBV	Infraestructura Banca Virtual
IVR	24Fono
CTE	Cuenta Corriente
AHO	Cuenta de Ahorros
CAM	Cambios

Tipo de datos

- ✓ Evitar la declaración de varchar(1), reemplazar por char(1).
- ✓ Evitar la declaración de varchar(2), reemplazar por char(2).
- ✓ Evaluar la declaración varchar(3) por char(3).
- ✓ Definir correctamente la longitud del campo de acuerdo al dato que se almacena, evitar el desperdicio de espacio en disco.

existe bit 0-1

- ✓ Tener mucho cuidado en el uso de tipo de dato varchar(max). Se debe justificar su uso. Consumo de memoria y cpu en ambientes altamente transaccionales.



Nombre de la tabla

- ✓ El nombre de la tabla debe tener una longitud máxima de 25 caracteres, debe ser significativo y en singular.
- ✓ Debe empezar con el nemónico de la aplicación + “_” + **tipo de tabla** + “_” + descripción de la tabla.
- ✓ Los valores permitidos para identificar el **tipo de tabla** son:

Tipos de tablas para Base de datos Transaccionales

Tipo/tabla	Usado en
m	Maestro
r	Relacional (usada para evitar tablas con relación muchos a muchos)
t	Transacción
c	Cabecera de Transacción (Factura)
d	Detalle de Transacción (Detalle de factura)
h	Históricos
p	Parámetro/catálogo
s	Secuenciales
l	Log de transacciones
o	Tablas de paso de información para procesos – BCP Normalmente no tienen índice

Nombre de la tabla

Tipos de tablas para Base de datos Analíticas

Tipo/tabla	Usado en
d	Dimensiones
h	Hecho
r	Datos de entrada para Excel y/o reporting services
o	Operaciones bancarias
t	Transacciones de operaciones
e	Datos de entidades externas
i	Datos internos por ejemplo: CRM
p	Plantilla

Ejemplo de Nombre de tabla

Tabla	Descripción
con_m_cuenta	Maestro de cuentas contables
aho_t_servicio	Transacciones de servicio de ahorros
aho_h_bloqueo	Histórico de transacciones de bloqueo
iva_c_factura_sri	Cabecera de factura
iva_d_factura_sri	Detalle de factura
int_l_eventos	Log de eventos de internet
dwh_d_oficiales	Dimensión de Oficiales

Creación de tabla

- ✓ La tabla de tipo transacción, cabecera y detalle son utilizadas para procesos OLTP diarios, la información no necesaria en línea debe a través de procesos BATCH pasar a tablas históricas.
- ✓ Toda tabla histórica debe ser depurada de acuerdo al tiempo de permanencia de los datos establecidos por el usuario final y las leyes vigentes del país.
- ✓ Al crear la tabla se debe indicar el dueño. Para los motores Sybase y Sql el dueño por corresponde a **dbo**.
- ✓ Para acceder a una tabla cuyo dueño es dbo se puede realizar de dos maneras:
 - Indicar en la instrucción SQL solo el nombre de la tabla `cl_ente`.
 - Indicar en la instrucción SQL el nombre de la base de datos y la tabla: `cobis..cl_ente`.
Evitar la definición `cobis.dbo.cl_ente`.
- ✓ Por legibilidad de código, se recomienda la definición de un campo por línea.
- ✓ El script de creación de tabla debe ser desarrollado considerando la reejecución.
- ✓ Todo script de base de datos que se compile en Sybase o SqlServer debe finalizar con la instrucción **go**. Sin **espacio** ni **TAB** antes ni después de su declaración.

Nombre de campos

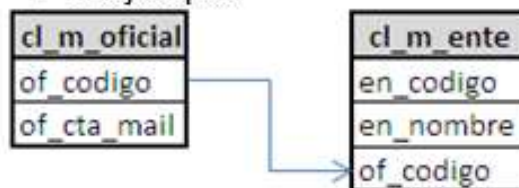
- ✓ Longitud máxima del nombre del campo 25 caracteres
- ✓ Debe empezar con las iniciales o prefijo (2 letras) de la tabla + "_" + descripción del campo.

Ejemplos:

ch_cuenta	}	cc_m_chequera
ch_chequera		
ch_fecha_emision		
.....		
tm_fecha	}	cc_h_tran_monet
tm_secuencial		
tm_cuenta		

- ✓ Al establecer una relación que permita mantener la integridad referencial entre tablas de aplicación NO COBIS, el campo debe mantener el nombre de la tabla que origina la relación

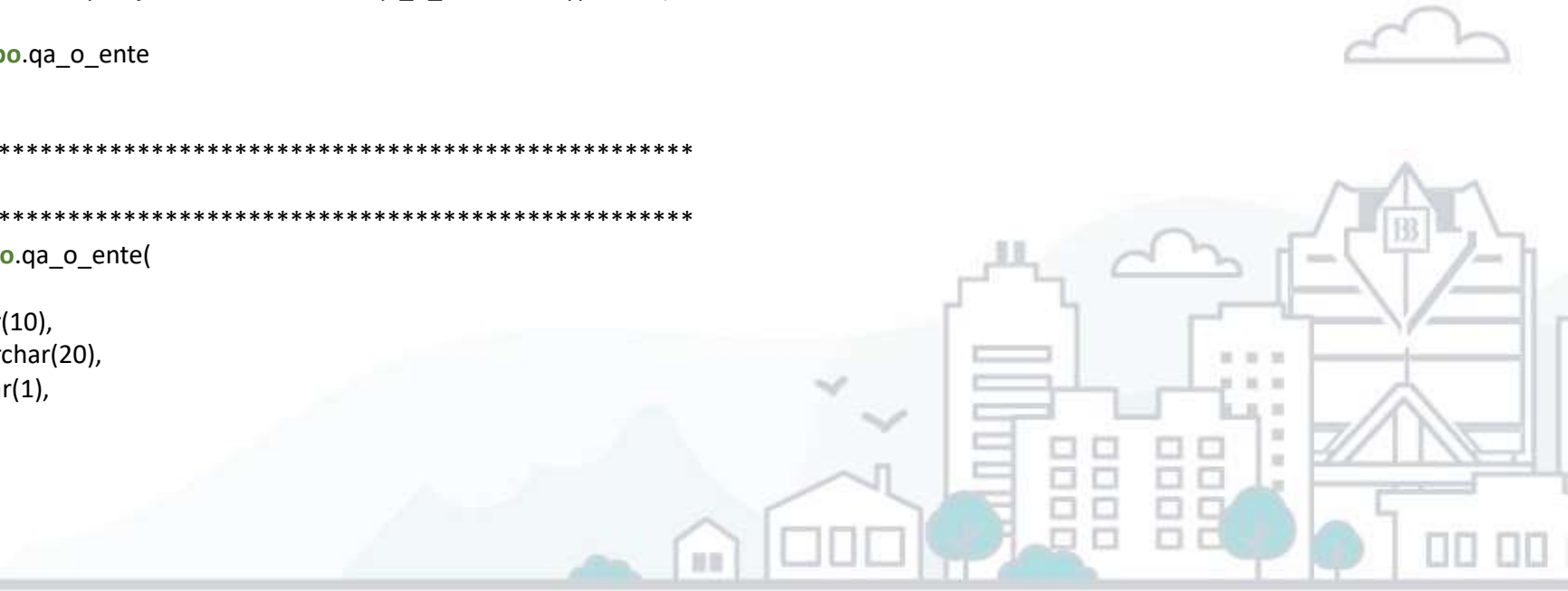
Por ejemplo:



Creación de tabla - Script

Algoritmo – Validar la existencia del objeto, eliminar el objeto y crear el objeto.

```
use cobis
go
set nocount on
-- *****
-- Eliminar la tabla
-- *****
if exists(select 1 from sysobjects where name = 'qa_o_ente' and type = 'U')
begin
    drop table dbo.qa_o_ente
end
go
-- *****
-- Crea tabla
-- *****
create table dbo.qa_o_ente(
    en_ente int,
    en_cedula char(10),
    en_nombre varchar(20),
    en_subtipo char(1),
    en_filial int,
    en_oficina int
)
go
```



Creación de tabla - Valores por defecto

Estructura Sybase

Column_name	Type	Length	Prec	Scale	Nulls
en_ente	int	4	NULL	NULL	0
en_cedula	char	10	NULL	NULL	0
en_nombre	varchar	20	NULL	NULL	0
en_subtipo	char	1	NULL	NULL	0
en_filial	int	4	NULL	NULL	0
en_oficina	int	4	NULL	NULL	0
en_fecing	datetime	8	NULL	NULL	0
en_usring	int	4	NULL	NULL	0

- ✓ Por defecto no se permite valores nulos en los campos.

en_ente	en_cedula	en_nombre	en_subtipo	en_filial	en_oficina	en_fecing	en_usring
1	0123456789	BOLIVARIANO CA	C	1	1	Oct 2 2021 5:33PM	1

Estructura SqlServer

Column_name	Type	Computed	Length	Prec	Scale	Nullable
en_ente	int	no	4	10	0	yes
en_cedula	char	no	10			yes
en_nombre	varchar	no	20			yes
en_subtipo	char	no	1			yes
en_filial	int	no	4	10	0	yes
en_oficina	int	no	4	10	0	yes
en_fecing	dateti...	no	8			yes
en_usring	int	no	4	10	0	yes

- ✓ Por defecto se permite el valor nulo en los campos.

[illegible]

Creación de tabla - Script

Algoritmo SYBASE – En caso de no perder datos por reejecución, crear la tabla dinámicamente solo cuando no exista el objeto.

```
use cobis
go
set nocount on
-- Valida existencia
if exists(select 1 from sysobjects where name = 'qa_o_ente' and type = 'U')
begin
    print 'tabla existe qa_o_ente'
end
else
begin
    -- Crea tabla dinámicamente
    execute('
    create table dbo.qa_o_ente(
    en_ente int,
    en_cedula char(10),
    en_nombre varchar(20),
    en_subtipo char(1),
    en_filial int,
    en_oficina int,
    en_fecing datetime,
    en_usring int
    )
    ')
end
go
```



Creación de tabla - Script

Algoritmo SQLSERVER – En caso de no perder datos por reejecución, crear la tabla solo cuando no exista.

```
use cobis
go
set nocount on
-- Valida existencia
if exists(select 1 from sysobjects where name = 'qa_o_ente' and type = 'U')
begin
    print 'tabla existe qa_o_ente'
end
else
begin
    -- Crea tabla
    create table dbo.qa_o_ente(
        en_ente int,
        en_cedula char(10),
        en_nombre varchar(20),
        en_subtipo char(1),
        en_filial int,
        en_oficina int,
        en_fecing datetime,
        en_usring int
    )
end
go
```



Nombre de la tabla temporal

- ✓ Este tipo de tabla utiliza la base de datos tempdb.
- ✓ El nombre de la tabla temporal debe tener una longitud máxima de 25 caracteres, descriptivo y en singular.
- ✓ La tabla debe empezar con el carácter "#" + nemónico de la aplicación + "_tmp_" + descripción de la tabla.
Ejemplo: #sat_tmp_orden
- ✓ En Sybase el límite actual para la base de datos tempdb es de 5GB de data y 3GB de log.



Creación de tabla temporal - Script

Algoritmo – Validar la existencia del objeto, eliminar el objeto y crear el objeto.

```
use cobis
go
set nocount on
if not object_id('tempdb..#sat_tmp_orden') is null
    drop table #sat_tmp_orden
go
create table #sat_tmp_orden (
orden int
)
go
```



Creación de tabla temporal – Error por transacción abierta

- ❖ En Sybase se debe evitar la creación de una tabla temporal cuando existe una transacción abierta, caso contrario se presentará un mensaje de error.

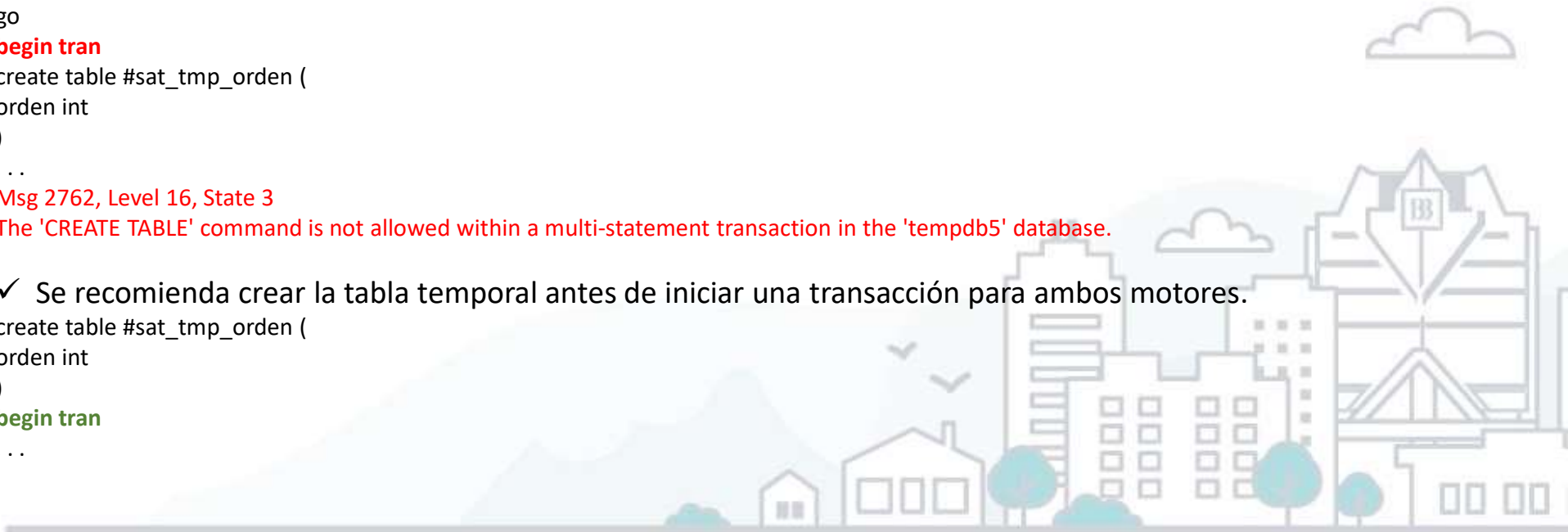
```
use cobis
go
set nocount on
if not object_id('tempdb..#sat_tmp_orden') is null
  drop table #sat_tmp_orden
go
begin tran
create table #sat_tmp_orden (
orden int
)
...
```

Msg 2762, Level 16, State 3

The 'CREATE TABLE' command is not allowed within a multi-statement transaction in the 'tempdb5' database.

- ✓ Se recomienda crear la tabla temporal antes de iniciar una transacción para ambos motores.

```
create table #sat_tmp_orden (
orden int
)
begin tran
...
```



Nombre de tabla física para uso temporal

- ✓ En SYBASE debido a que los procesos BATCH procesan grandes volúmenes de datos (millones de registros o mínimo 500 MB de datos por ejecución de cada proceso), existen dos bases de datos db_general y db_temporal con espacio suficiente para que no exista novedad de espacio ni log suspend.
- ✓ El espacio disponible en el servidor de producción es:

Base de datos	Área de Datos	Área de Log
db_general	12 GB	6 GB
db_temporal	10 GB	8 GB

- ✓ El nombre de la tabla temporal debe tener una longitud máxima de 25 caracteres, descriptivo y en singular.
- ✓ La nomenclatura de la tabla fija temporales es:
Nemónico de la aplicación + "_tmp_" + descripción de la tabla.

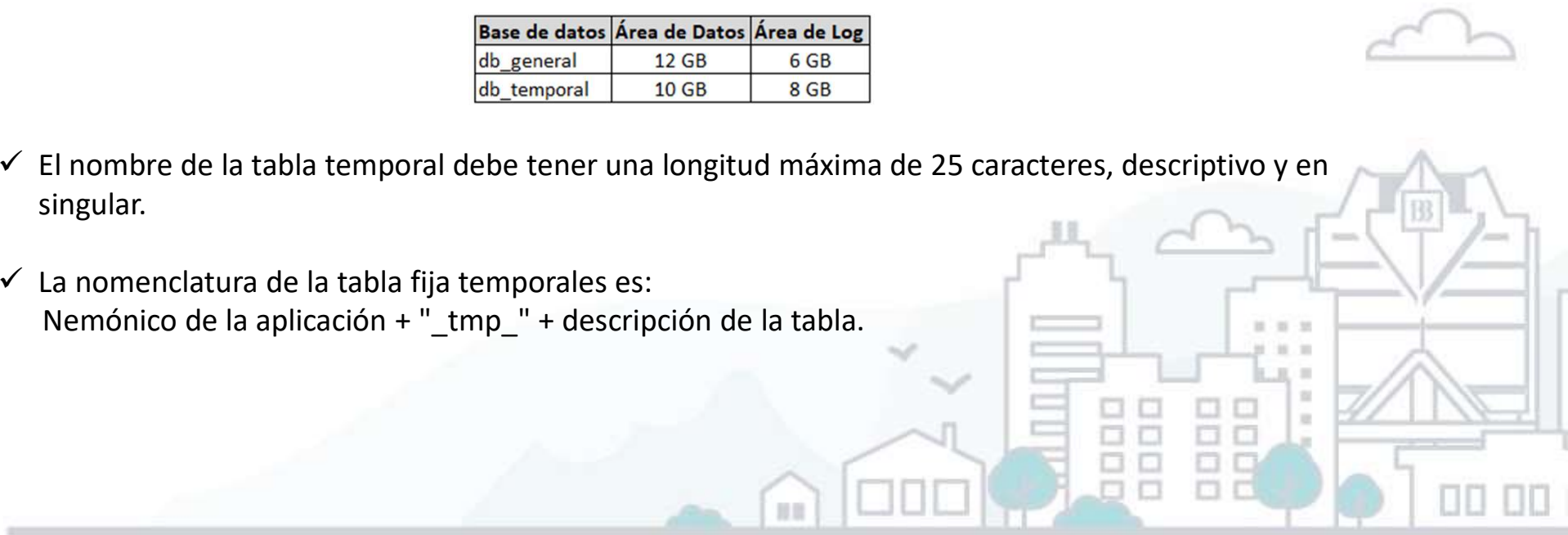
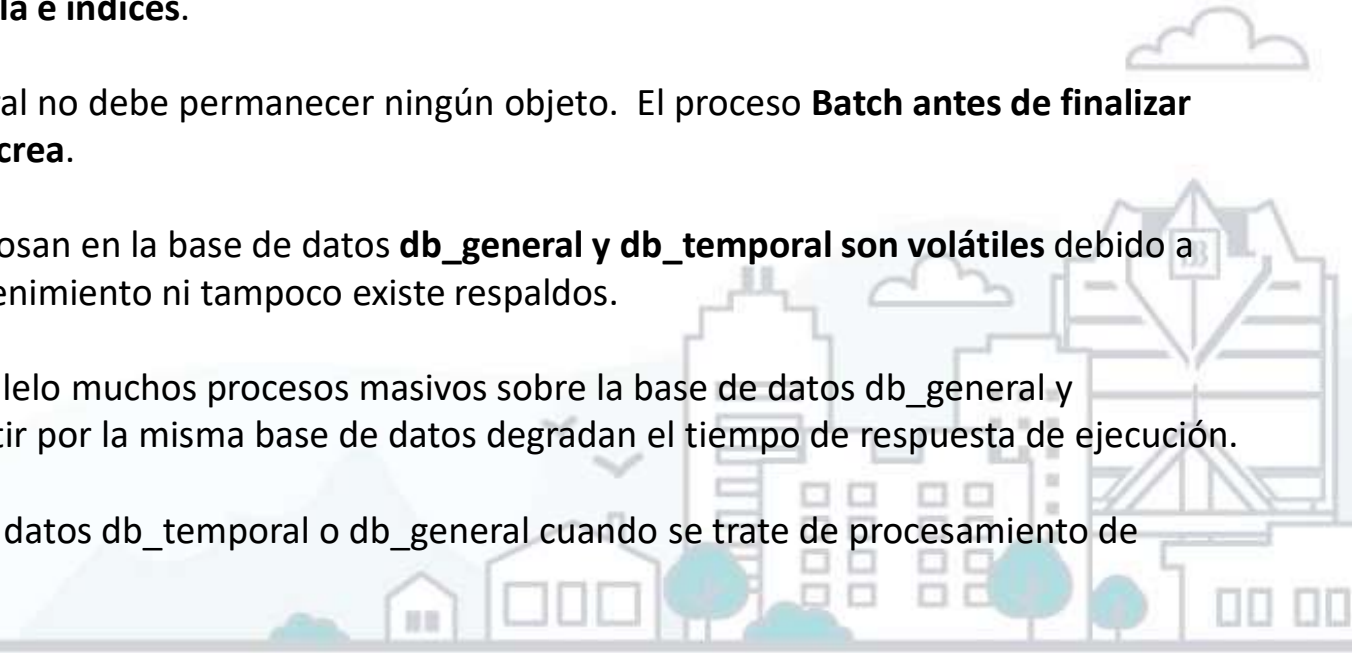


Tabla física de uso temporal - Consideraciones y restricciones

Consideraciones y restricciones al usar la base de datos db_general y db_temporal

- ✓ En la base de datos **db_general** se debe **evitar dejar pobladas las tablas** a excepción que sea una tabla de pocos registros (menos de 1.000 registros) y que sea truncada cada vez que inicia/finaliza un nuevo proceso.
 - ✓ En la base de datos **db_temporal/db_general** no se compilan funciones, ni vistas ni procedimientos almacenados. **Solo se crea tabla e índices.**
 - ✓ En la base de datos db_temporal no debe permanecer ningún objeto. El proceso **Batch antes de finalizar debe eliminar los objetos que crea.**
 - ✓ Los **datos** de las tablas que reposan en la base de datos **db_general y db_temporal son volátiles** debido a que no existen tareas de mantenimiento ni tampoco existe respaldos.
 - ✓ Se debe evitar ejecutar en paralelo muchos procesos masivos sobre la base de datos db_general y db_temporal porque al competir por la misma base de datos degradan el tiempo de respuesta de ejecución.
 - ✓ Solo se debe utilizar la base de datos db_temporal o db_general cuando se trate de procesamiento de millones de registros.
- 

Constraint – Primary Key

- ✓ La longitud máxima del nombre de constraint permitido es de 30 caracteres.
- ✓ Iniciará con la constante “pk_” + nombre de la tabla.
- ✓ No se permite índice compuesto como clave primaria, se debe definir como constraint.
- ✓ El campo seleccionado de preferencia debe ser numérico.

Ejemplo: pk_con_m_cuenta

Consideraciones:

- ✓ Toda tabla debe tener una clave primaria para identificar un único registro.
- ✓ La clave primaria puede estar conformada por un campo o varios campos .
- ✓ Los campos de la clave primaria deben estar definidos en la tabla como valores no nulos.
- ✓ No es obligatorio la definición de clave primaria aquellas tablas que son usadas para paso de datos.

Normalmente usada en BCP.

- ✓ La tabla solo puede contener una clave primaria.

La tabla es nueva:

```
create table qa_m_ente(  
id_ente int not null constraint pk_qa_m_ente primary key,  
...  
nombre varchar(100),  
fecing datetime  
)
```

La tabla existe:

```
alter table qa_m_ente  
add constraint pk_qa_m_ente  
primary key (id_ente)
```



Constraint – Foreign Key

- ✓ La longitud máxima del nombre de constraint permitido es de 30 caracteres.
- ✓ Iniciará con la constante “fk_” + abreviatura de la tabla origen + “_” + abreviatura de la tabla referenciada.
Ejemplo: fk_tcabfact_mclient

```
alter table qa_m_direccion  
add constraint fk_mdirec_mente  
foreign key (id_ente)  
references qa_m_ente (id_ente)
```



Constraint – Unique

- ✓ La longitud máxima del nombre de constraint permitido es de 30 caracteres.
- ✓ Iniciaré con la constante "uq_" + abreviatura de la tabla + "_" + abreviatura del campo.
- ✓ Se puede definir muchas restricciones unique.

```
alter table bb_m_cliente  
  add constraint uq_cliente_cedula  
  unique (cedula)
```

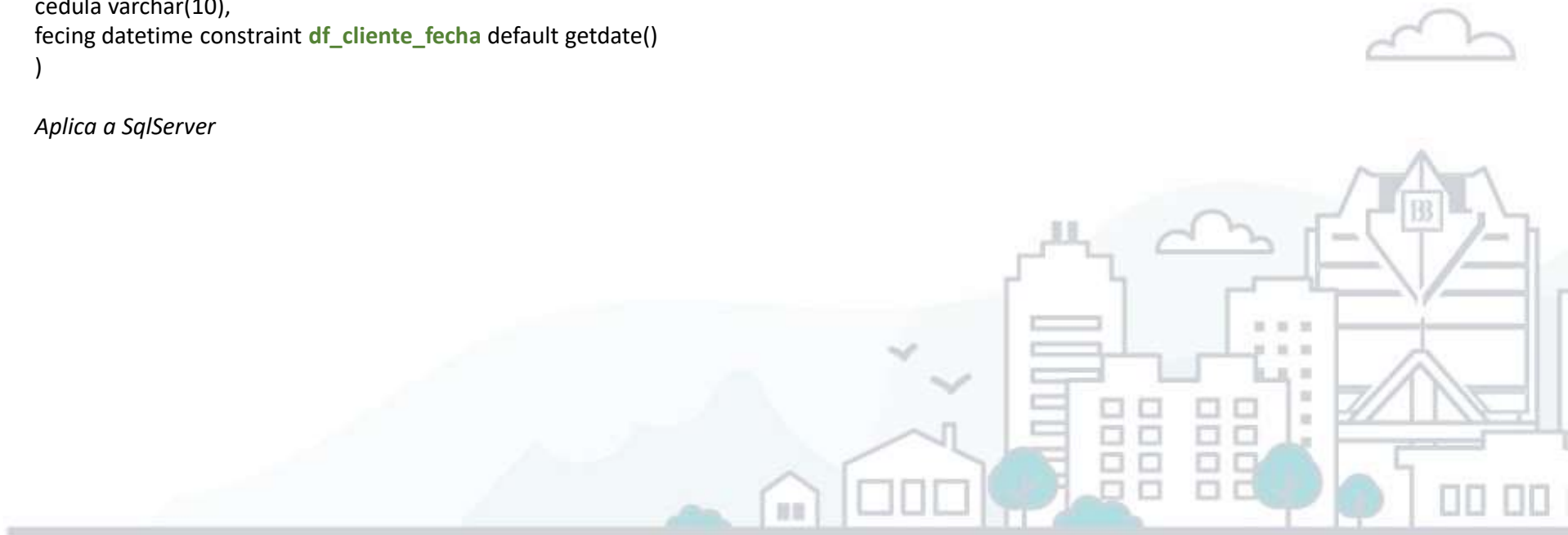


Constraint – Default

- ✓ La longitud máxima del nombre de constraint permitido es de 30 caracteres.
- ✓ Iniciará con la constante “df_” + abreviatura de la tabla + “_” + abreviatura del campo.

```
create table bb_m_cliente(  
  id_cliente int,  
  nombre varchar(100),  
  cedula varchar(10),  
  fecing datetime constraint df_cliente_fecha default getdate()  
)
```

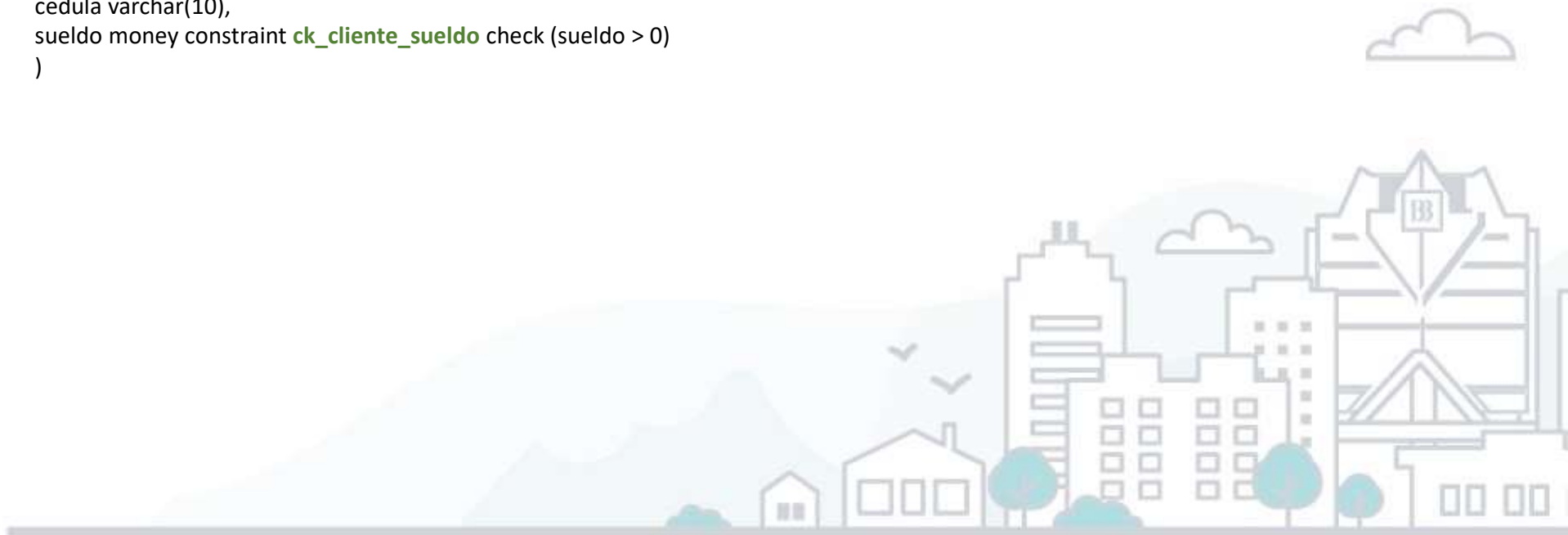
Aplica a SqlServer



Constraint – Check

- ✓ La longitud máxima del nombre de constraint permitido es de 30 caracteres.
- ✓ Iniciará con la constante "ck_" + abreviatura de la tabla + "_" + abreviatura del campo.

```
create table bb_m_cliente(  
  id_cliente int,  
  nombre varchar(100),  
  cedula varchar(10),  
  sueldo money constraint ck_cliente_sueldo check (sueldo > 0)  
)
```



Nombre de Índices

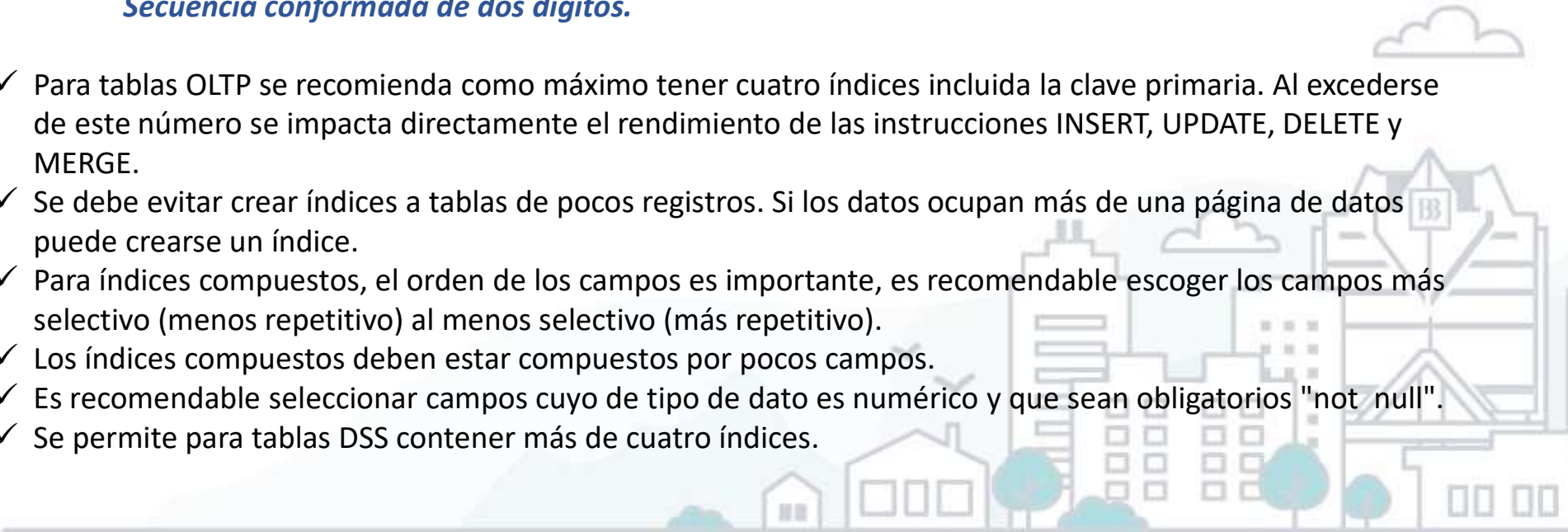
- ✓ La longitud máxima del nombre de los índices es de 30 caracteres.
- ✓ Los índices deben empezar con la constante "i_" + nombre de la tabla + "_" + secuencia

Ejemplos:

i_con_m_cuentas_01

i_aho_m_cliente_01

Secuencia conformada de dos dígitos.

- ✓ Para tablas OLTP se recomienda como máximo tener cuatro índices incluida la clave primaria. Al excederse de este número se impacta directamente el rendimiento de las instrucciones INSERT, UPDATE, DELETE y MERGE.
 - ✓ Se debe evitar crear índices a tablas de pocos registros. Si los datos ocupan más de una página de datos puede crearse un índice.
 - ✓ Para índices compuestos, el orden de los campos es importante, es recomendable escoger los campos más selectivo (menos repetitivo) al menos selectivo (más repetitivo).
 - ✓ Los índices compuestos deben estar compuestos por pocos campos.
 - ✓ Es recomendable seleccionar campos cuyo tipo de dato es numérico y que sean obligatorios "not null".
 - ✓ Se permite para tablas DSS contener más de cuatro índices.
- 

Índices – consideraciones especiales

- ✓ El índice clustered
 - ✓ Los datos están ordenados físicamente.
 - ✓ Es más rápido el acceso.
 - ✓ Una tabla solo se puede definir un índice clustered.
 - ✓ Si es altamente transaccional insert/update es posible generar deadlock.
- ✓ El índice non-clustered.
 - ✓ Es más lento su acceso porque primero lee la página de índice para ubicar la dirección de la fila y luego accede a la página de datos para extraerlo.
- ✓ Si una tabla realiza muchas operaciones de inserción, actualización o eliminación corren el riesgo que las estadísticas de estar desactualizadas por ende el optimizador no tomará un buen plan de ejecución y puede causar lentitud.
- ✓ Cada vez que se crea un índice se debe actualizar las estadísticas:

Sybase

```
update index statistics bb_m_cliente  
exec sp_recompile bb_m_cliente
```

SqlServer

```
update statistics bb_m_cliente  
exec sp_recompile bb_m_cliente
```

Índices – consideraciones especiales

- ✓ Antes de crear un índice se sugiere analizar la distribución de los datos. Por ejemplo:

Cantidad total de registros de la tabla: → 320.000 100%

Calcular la cantidad máxima de valor repetido por el o los campos agrupados que son candidatos al nuevo índice: → 656

```
select max(cnt) cnt
from
(
    select campo, count(1) cnt
    from tabla
    group by campo
) valida
```

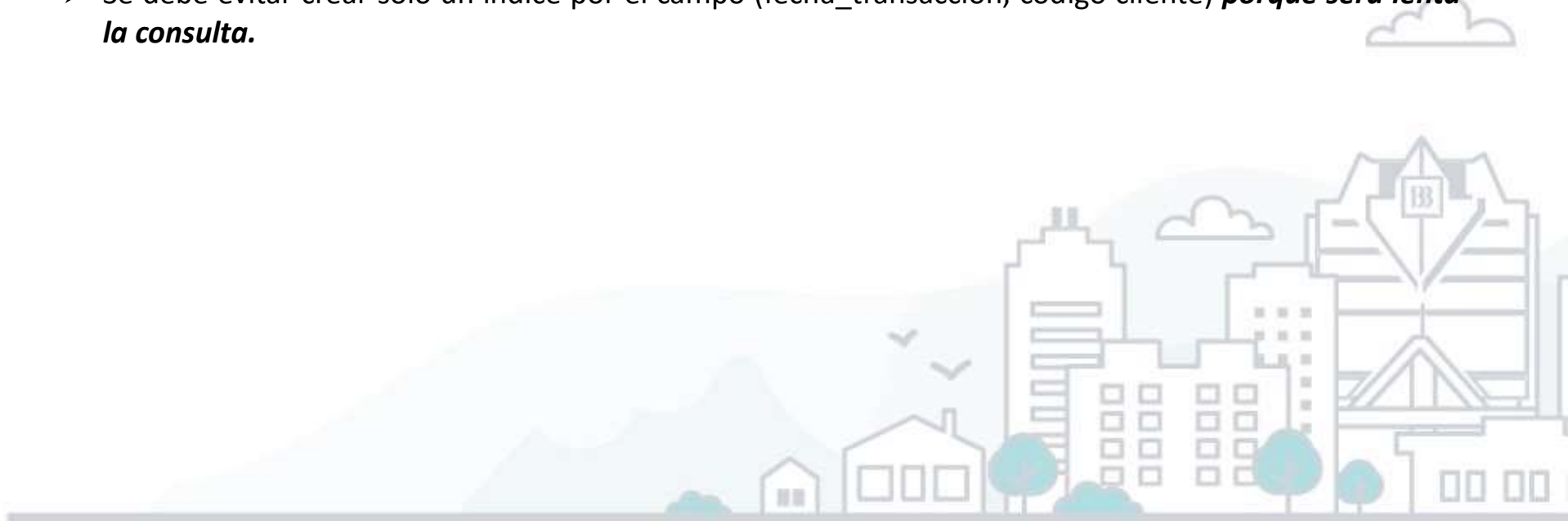
Establecer el porcentaje que corresponde a la cantidad máxima de registros agrupados repetidos. Si es menos del 10 % se considera viable crear el índice. $(656 \times 100) / 320.000 = 0.205 \%$

- ✓ Se debe evitar crear índices por campos estado. Se ha constatado que el optimizador no utiliza este tipo de índice.



Índices – consideraciones especiales

- ✓ En el siguiente escenario, existen dos tipos de consulta: consulta por transacciones del cliente ordenado por fecha y consulta por una fecha o rango de fechas:
 - Para la primera consulta es eficiente crear índice por los campos: (código cliente, fecha_transacción)
 - Para la segunda consulta es eficiente crear índice el campo fecha: (fecha_transacción)
 - Se debe evitar crear solo un índice por el campo (fecha_transacción, código cliente) **porque será lenta la consulta.**



Sentencia Select

- ✓ Toda sentencia Select debe ser legible su lectura, cumplir con el siguiente esquema:

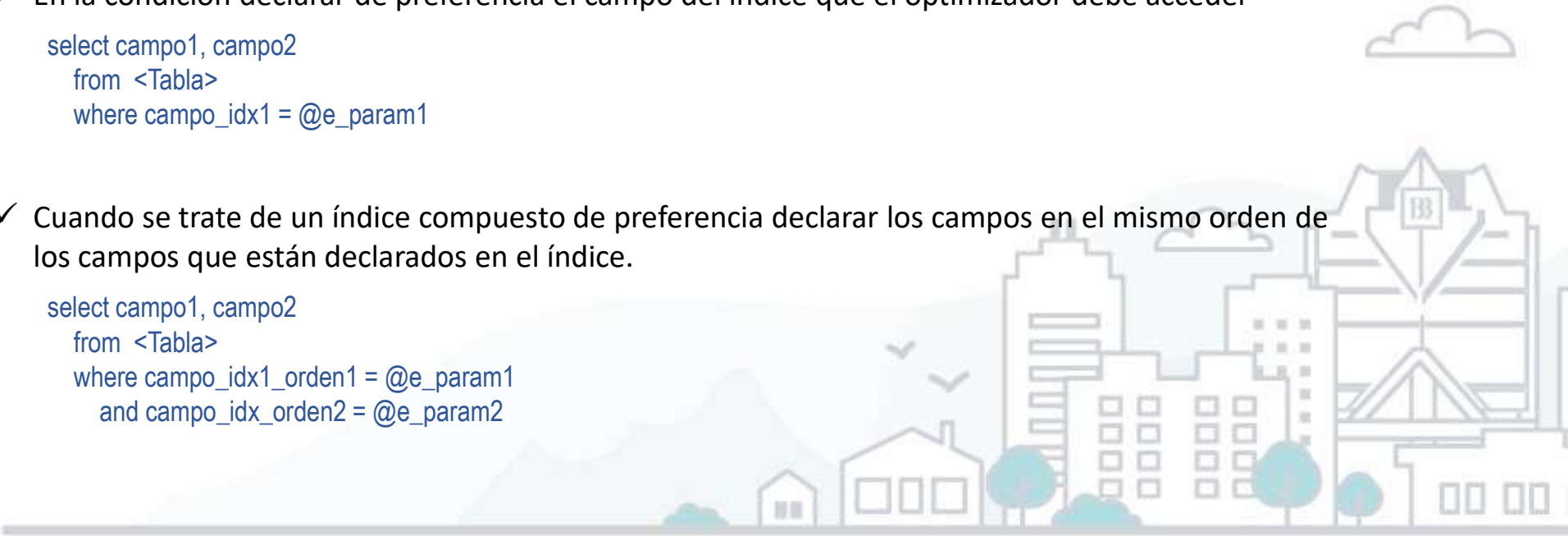
```
select campo1, campo2  
  from <Tabla>  
 where <condiciones>
```

- ✓ En la condición declarar de preferencia el campo del índice que el optimizador debe acceder

```
select campo1, campo2  
  from <Tabla>  
 where campo_idx1 = @e_param1
```

- ✓ Cuando se trate de un índice compuesto de preferencia declarar los campos en el mismo orden de los campos que están declarados en el índice.

```
select campo1, campo2  
  from <Tabla>  
 where campo_idx1_orden1 = @e_param1  
       and campo_idx_orden2 = @e_param2
```



Estándares ANSI SQL

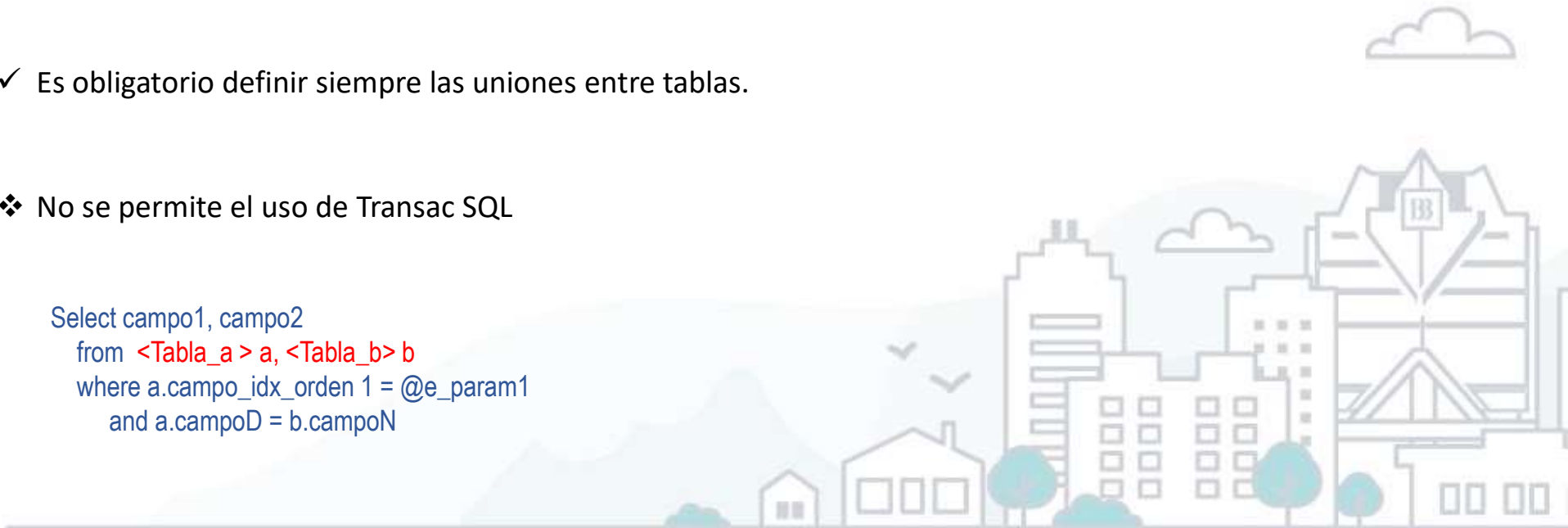
- ✓ Toda sentencia debe cumplir el estándar ANSI SQL.

```
Select campo1, campo2
from <Tabla_a> a,
inner join <Tabla_b> b
on a.campo = b.campo_idx_orden1
where a.campo_idx_orden 1 = @e_param1
```

- ✓ Es obligatorio definir siempre las uniones entre tablas.

- ❖ No se permite el uso de Transac SQL

```
Select campo1, campo2
from <Tabla_a> a, <Tabla_b> b
where a.campo_idx_orden 1 = @e_param1
and a.campoD = b.campoN
```

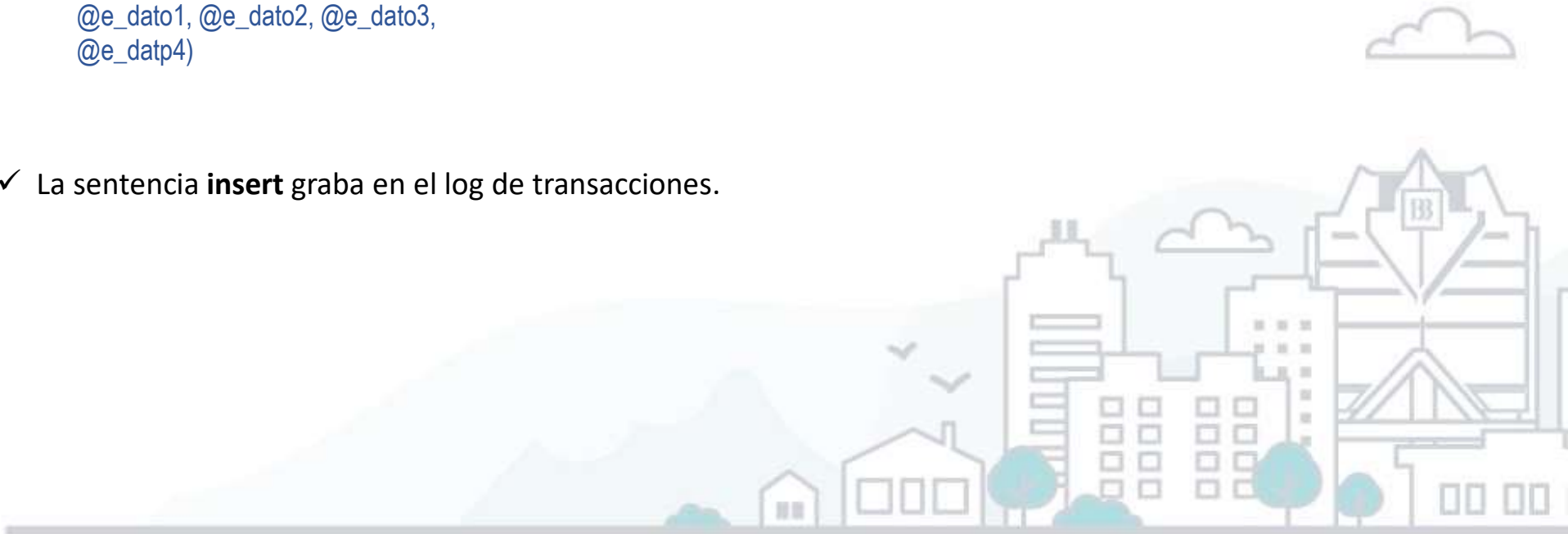


Sentencia Insert

- ✓ Toda sentencia Insert debe ser legible su lectura, cumplir con el siguiente esquema:

```
Insert into <Tabla>(
    campo1, campo2, campo3,
    campo4
)
values(
    @e_dato1, @e_dato2, @e_dato3,
    @e_datp4)
```

- ✓ La sentencia **insert** graba en el log de transacciones.

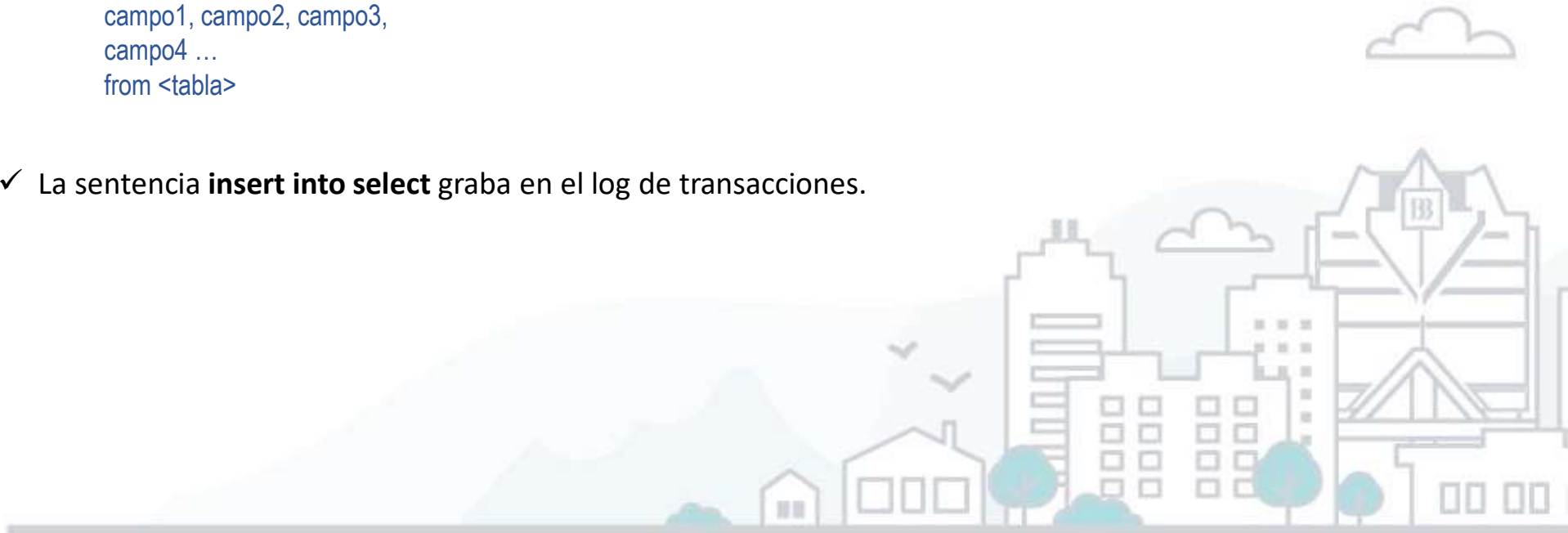


Sentencia Insert ... select

- ✓ Toda sentencia Insert into ... select debe ser legible su lectura, cumplir con el siguiente esquema:

```
Insert into <Tabla>(  
    campo1, campo2, campo3,  
    campo4 ...  
)  
select  
    campo1, campo2, campo3,  
    campo4 ...  
from <tabla>
```

- ✓ La sentencia **insert into select** graba en el log de transacciones.



Sentencia Update

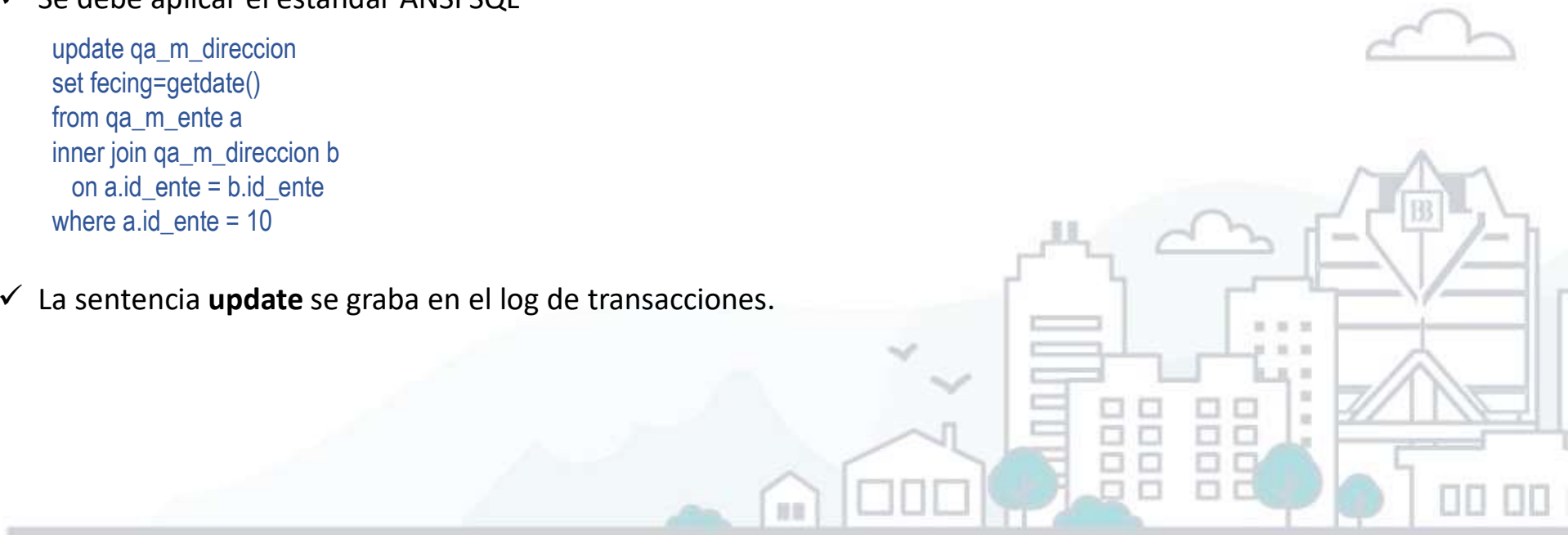
- ✓ Toda sentencia Update debe ser legible su lectura, cumplir con el siguiente esquema:

```
update <Tabla>  
  set <campo> = <valor>  
  where <condiciones>
```

- ✓ Se debe aplicar el estándar ANSI SQL

```
update qa_m_direccion  
set fecing=getdate()  
from qa_m_ente a  
inner join qa_m_direccion b  
  on a.id_ente = b.id_ente  
where a.id_ente = 10
```

- ✓ La sentencia **update** se graba en el log de transacciones.



Sentencia Delete

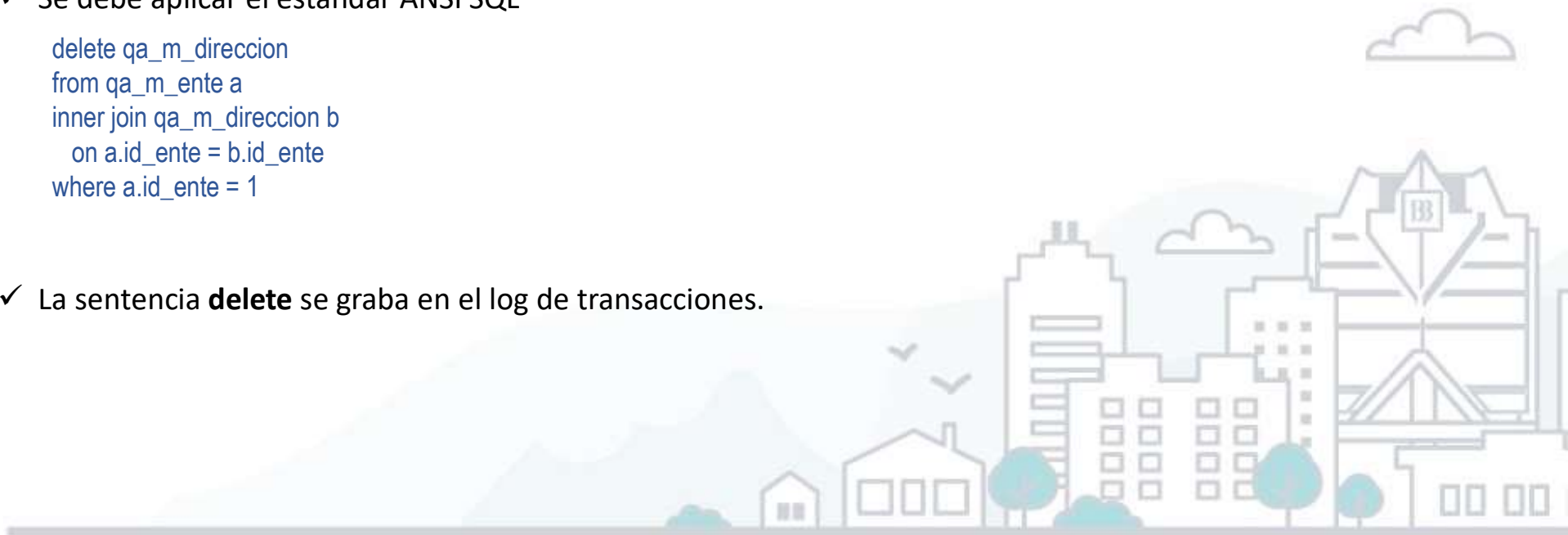
- ✓ Toda sentencia Update debe ser legible su lectura, cumplir con el siguiente esquema:

```
delete <Tabla>  
where <condiciones>
```

- ✓ Se debe aplicar el estándar ANSI SQL

```
delete qa_m_direccion  
from qa_m_ente a  
inner join qa_m_direccion b  
  on a.id_ente = b.id_ente  
where a.id_ente = 1
```

- ✓ La sentencia **delete** se graba en el log de transacciones.

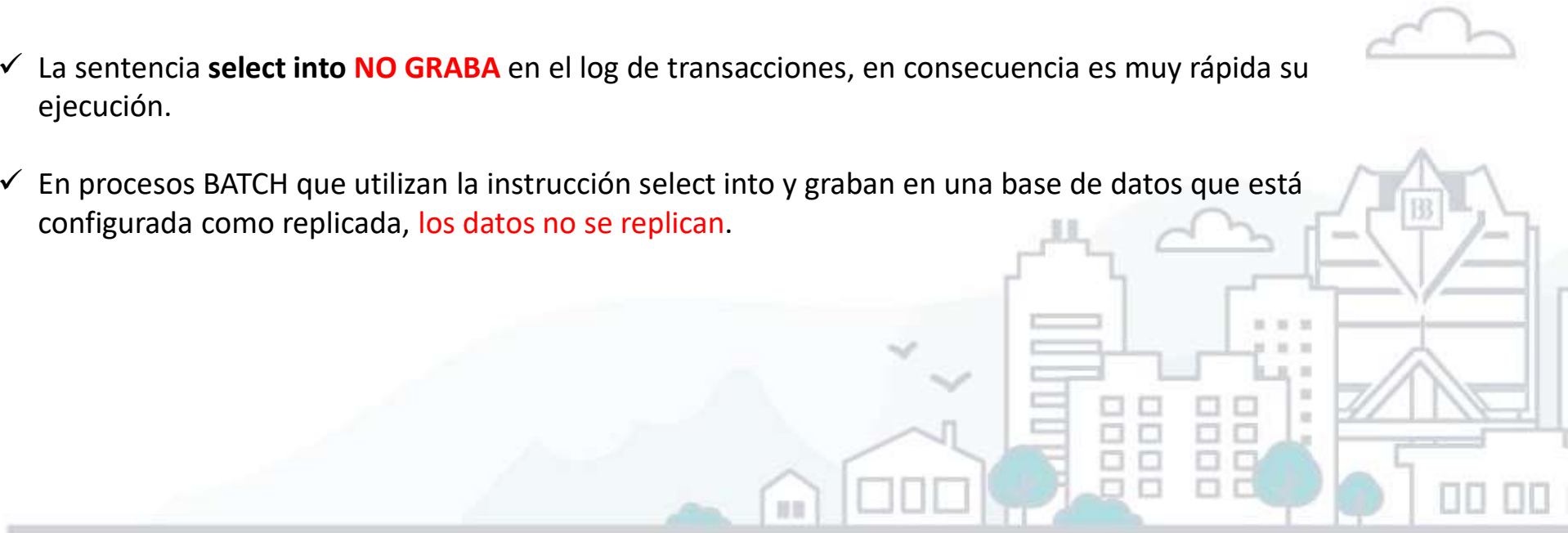


Sentencia select into

- ✓ Toda sentencia select into debe ser legible su lectura, cumplir con el siguiente esquema:

```
select  
  campo1, campo2, campo3,  
  campo4 ...  
into <table_nueva>  
from <tabla>
```

- ✓ La sentencia **select into NO GRABA** en el log de transacciones, en consecuencia es muy rápida su ejecución.
- ✓ En procesos BATCH que utilizan la instrucción select into y graban en una base de datos que está configurada como replicada, **los datos no se replican**.



Lógica que se debe evitar

- ❖ Evitar invocar funciones en las sentencias porque consumen cpu.

```
Select campo1, campo2, func_calculo(campo3)
  from <Tabla>
 where <condiciones>
```

- ❖ Evitar utilizar la sentencia select *, se debe describir los campos.

- ❖ Evitar evaluar en la condición variables con valores fijos
where @v_opción = 1 and monto > 10

- ❖ Evitar evaluar en la condición parámetros con valores fijos
where @e_accion = 'A'

- ❖ Evitar el uso de funciones a los campos indexados, el optimizador genera table scan.

```
where substring (campo_idx_orden1, 2, 3) = 10
```

```
where upper(campo_idx_orden1) = 'AAA'
```

```
where isnull (campo_idx_orden1, 'A') = 'A'
```



Lógica que se debe evitar

- ❖ Si la consulta accede por un índice compuesto, en la condición debe describirse los campos que lo conforman caso contrario el optimizador no lo utiliza.

Índice compuesto: campo_a, campo_b, campo_c

El optimizador si usa el índice:

Where **campo_a** = @e_param1 and **campo_b** = @e_param2 and **campo_c** = @e_param3

El optimizador no usa el índice, realiza un table scan

Where campo_b = @e_param2 and campo_c = @e_param3

Where campo_c = @e_param3

En esta asignación el optimizador realiza un table scan.

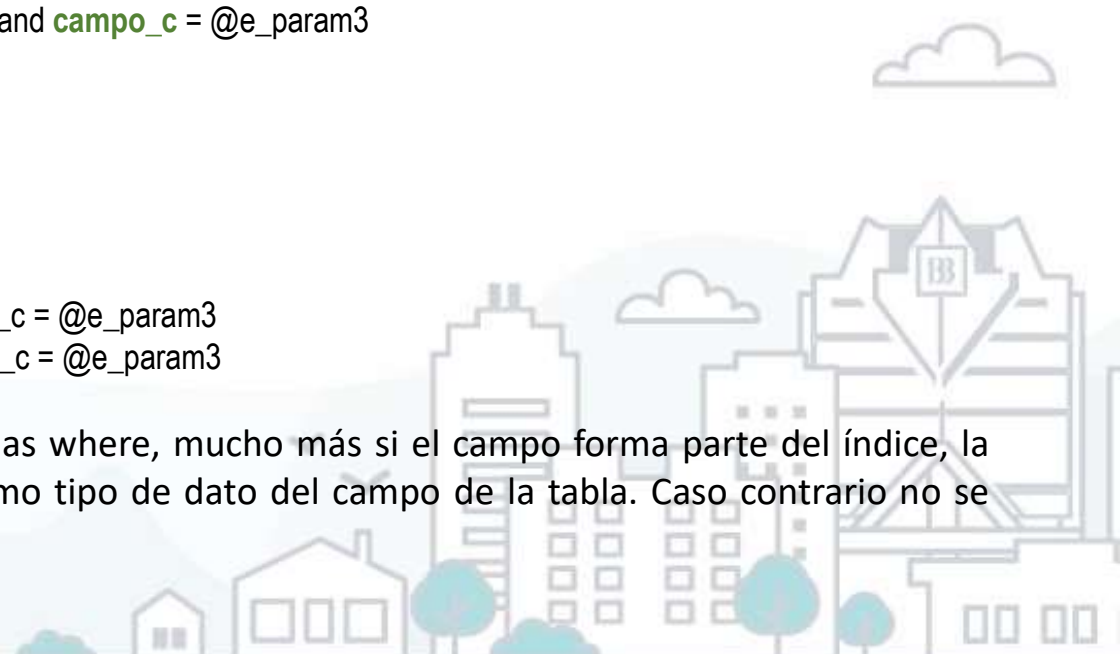
Where **campo_a** >= " and campo_b = @e_param2 and campo_c = @e_param3

Where **campo_a** >= 0 and campo_b = @e_param2 and campo_c = @e_param3

- ❖ Evitar las conversiones de datos en las cláusulas where, mucho más si el campo forma parte del índice, la variable con la cual se filtra debe ser del mismo tipo de dato del campo de la tabla. Caso contrario no se estaría usando el índice, ejemplo:

Declare @campo char(10)

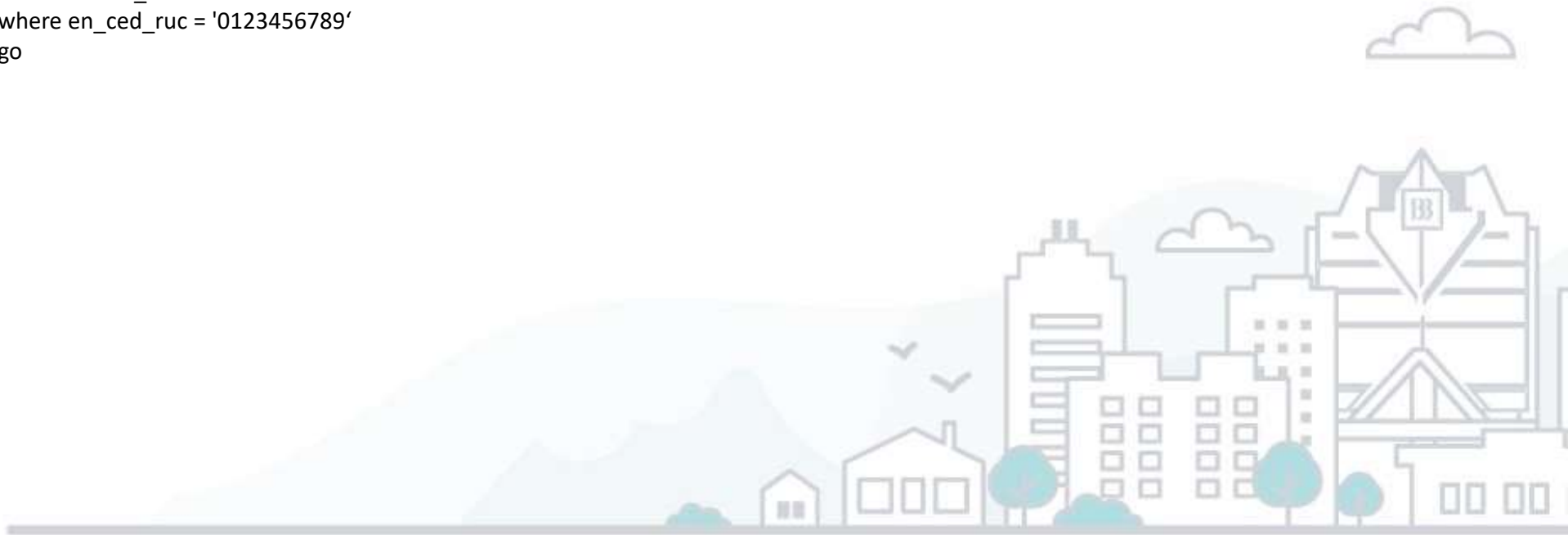
Select Where **convert(int, campo_indice)** = @e_campo



Plan de ejecución y consumo de IO

- ✓ Para obtener el Plan de ejecución y el consumo de IO en Sybase es necesario habilitarlo:

```
set showplan on
set statistics io on
go
select en_ente, en_nombre, en_subtipo, en_filial, en_oficina, en_ced_ruc, en_fecha_crea
from cobis..cl_ente a
where en_ced_ruc = '0123456789'
go
```



QP/IO

```
select en_ente, en_nombre, en_subtipo, en_filial, en_oficina, en_ced_ruc, en_fecha_crea
from cobis..cl_ente a
where en_ced_ruc = '0123456789'
```

STEP 1 → # de Línea

The type of query is SELECT.

1 operator(s) under root

| ROOT:EMIT Operator (VA = 1)

|

| | SCAN Operator (VA = 0)

| | FROM TABLE

| | cobis..cl_ente

| | a

| | Index : ien_cedruc

| | Forward Scan.

| | Positioning by key.

| | Keys are:

| | en_ced_ruc ASC

| | Using I/O Size 8 Kbytes for index leaf pages.

| | With LRU Buffer Replacement Strategy for index leaf pages.

| | Using I/O Size 8 Kbytes for data pages.

| | With LRU Buffer Replacement Strategy for data pages.

Table: cobis..cl_ente (a) scan count 1, logical reads: (regular=4 apf=0 total=4), physical reads: (regular=0 apf=0 total=0), apf IOs used=0
Total writes for this command: 0

QP/IO

```
select en_ente, en_nombre, en_subtipo, en_filial, en_oficina, en_ced_ruc, en_fecha_crea
from cobis..cl_ente a
where en_ced_ruc = '0123456789' or p_pasaporte = '0123456789'
```

```
| ROOT:EMIT Operator (VA = 6)
|
| | RID JOIN Operator (VA = 5)
| | | Using Worktable2 for internal storage.
| |
| | | HASH UNION Operator (VA = 2) has 2 children.
| | | Using Worktable1 for internal storage.
| | | Key Count: 1
| |
| | | SCAN Operator (VA = 0)
| | | FROM TABLE
| | | cobis..cl_ente
| | | a
| | | Index : ien_cedruc
| | | Forward Scan.
| | | Positioning by key.
| | | Index contains all needed columns. Base table will not be read.
| | | Keys are:
| | | en_ced_ruc ASC
| | | Using I/O Size 8 Kbytes for index leaf pages.
| | | With LRU Buffer Replacement Strategy for index leaf pages.
```



QP/IO

where en_ced_ruc = '0123456789' or p_pasaporte = '0123456789'

| | | SCAN Operator (VA = 1)
| | | FROM TABLE
| | | cobis..cl_ente
| | | a
| | | Index : **ien_pasapote**
| | | Forward Scan.
| | | Positioning by key.
| | | Index contains all needed columns. Base table will not be read.
| | | Keys are:
| | | **p_pasaporte ASC**
| | | Using I/O Size 8 Kbytes for index leaf pages.
| | | With LRU Buffer Replacement Strategy for index leaf pages.

| | | RESTRICT Operator (VA = 4)(0)(0)(0)(7)(0)

| | | SCAN Operator (VA = 3)
| | | FROM TABLE
| | | cobis..cl_ente
| | | a
| | | **Using Dynamic Index.**
| | | Forward Scan.
| | | Positioning by Row Identifier (RID).
| | | Using I/O Size 8 Kbytes for data pages.
| | | With LRU Buffer Replacement Strategy for data pages.



QP/IO

where en_ced_ruc = '0123456789' or p_pasaporte = '0123456789'

Table: **Worktable2** scan count 1, logical reads: (regular=5 apf=0 total=5), physical reads: (regular=0 apf=0 total=0), apf IOs used=0

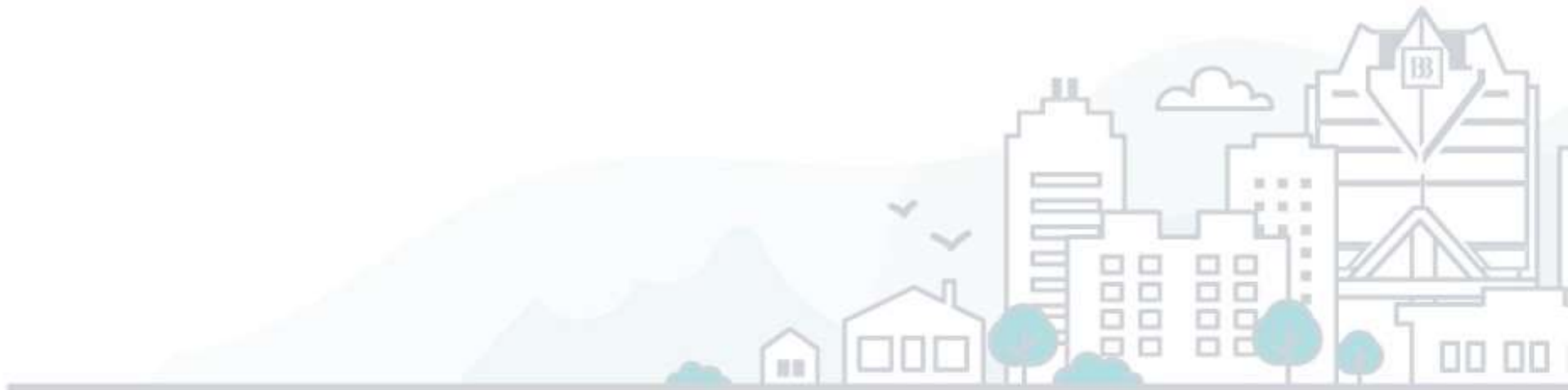
Table: **Worktable1** scan count 1, logical reads: (regular=2 apf=0 total=2), physical reads: (regular=0 apf=0 total=0), apf IOs used=0

Table: cobis..cl_ente (a) scan count 1, logical reads: (regular=3 apf=0 total=3), physical reads: (regular=0 apf=0 total=0), apf IOs used=0

Table: cobis..cl_ente (a) scan count 1, logical reads: (regular=3 apf=0 total=3), physical reads: (regular=0 apf=0 total=0), apf IOs used=0

Table: cobis..cl_ente (a) scan count 1, logical reads: (regular=1 apf=0 total=1), physical reads: (regular=0 apf=0 total=0), apf IOs used=0

Total writes for this command: 4



QP/IO

```
select en_ente, en_nombre, en_subtipo, en_filial, en_oficina,  
from cobis..cl_ente a  
where en_ced_ruc = '0123456789'  
union  
select en_ente, en_nombre, en_subtipo, en_filial, en_oficina,  
from cobis..cl_ente a  
where p_pasaporte = '0123456789'
```

ROOT:EMIT Operator (VA = 3)

HASH UNION Operator (VA = 2) has 2 children.

Using Worktable1 for internal storage.

Key Count: 7

SCAN Operator (VA = 0)

FROM TABLE

cobis..cl_ente

a

Index : ien_cedruc

Forward Scan.

Positioning by key.

Keys are:

en_ced_ruc ASC

Using I/O Size 8 Kbytes for index leaf pages.

With LRU Buffer Replacement Strategy for index leaf pages.

Using I/O Size 8 Kbytes for data pages.

With LRU Buffer Replacement Strategy for data pages.

SCAN Operator (VA = 1)

FROM TABLE

cobis..cl_ente

a

Index : ien_pasapote

Forward Scan.

Positioning by key.

Keys are:

p_pasaporte ASC

Using I/O Size 8 Kbytes for index leaf pages.

With LRU Buffer Replacement Strategy for index leaf pages.

Using I/O Size 8 Kbytes for data pages.

With LRU Buffer Replacement Strategy for data pages.

QP / IO

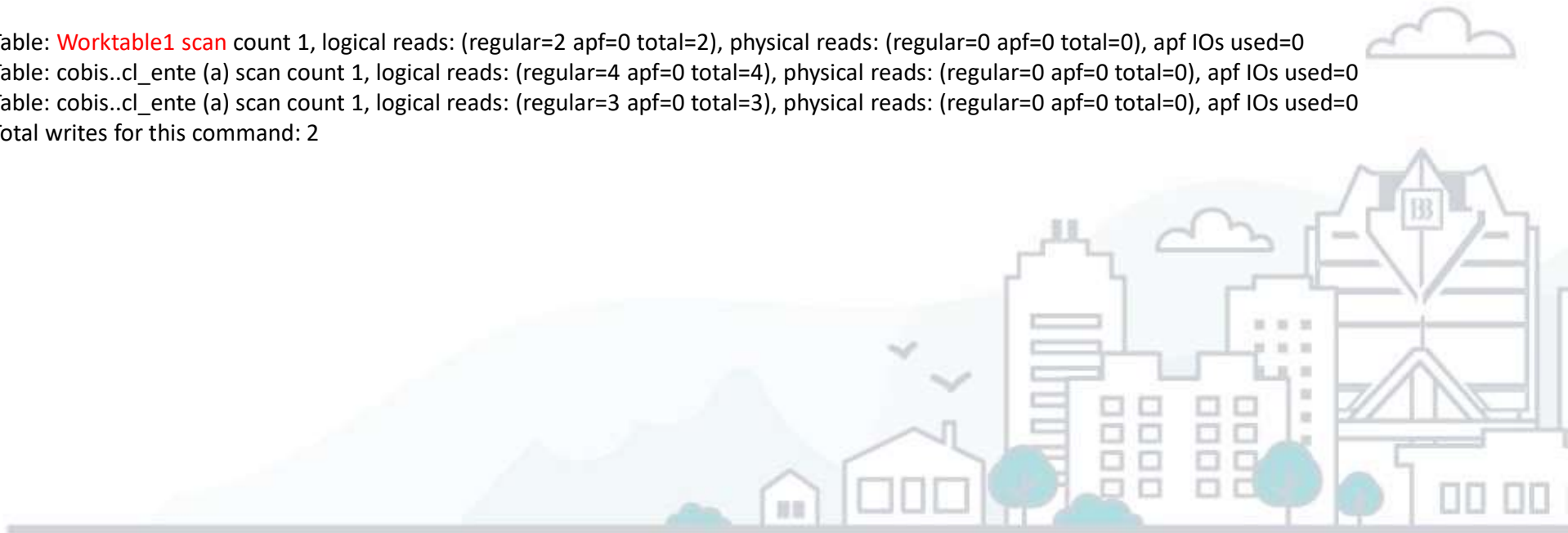
```
select en_ente, en_nombre, en_subtipo, en_filial, en_oficina, en_ced_ruc, en_fecha_crea
from cobis..cl_ente a
where en_ced_ruc = '0123456789'
union
select en_ente, en_nombre, en_subtipo, en_filial, en_oficina, en_ced_ruc, en_fecha_crea
from cobis..cl_ente a
where p_pasaporte = '0123456789'
```

Table: **Worktable1 scan** count 1, logical reads: (regular=2 apf=0 total=2), physical reads: (regular=0 apf=0 total=0), apf IOs used=0

Table: cobis..cl_ente (a) scan count 1, logical reads: (regular=4 apf=0 total=4), physical reads: (regular=0 apf=0 total=0), apf IOs used=0

Table: cobis..cl_ente (a) scan count 1, logical reads: (regular=3 apf=0 total=3), physical reads: (regular=0 apf=0 total=0), apf IOs used=0

Total writes for this command: 2



QP/IO

```
select en_ente, en_nombre, en_subtipo, en_filial, en_oficina,  
from cobis..cl_ente a  
where en_ced_ruc = '0123456789'  
union all  
select en_ente, en_nombre, en_subtipo, en_filial, en_oficina,  
from cobis..cl_ente a  
where p_pasaporte = '0123456789'
```

ROOT:EMIT Operator (VA = 3)

UNION ALL Operator (VA = 2) has 2 children.

SCAN Operator (VA = 0)

FROM TABLE

cobis..cl_ente

a

Index : ien_cedruc

Forward Scan.

Positioning by key.

Keys are:

en_ced_ruc ASC

Using I/O Size 8 Kbytes for index leaf pages.

With LRU Buffer Replacement Strategy for index leaf pages.

Using I/O Size 8 Kbytes for data pages.

With LRU Buffer Replacement Strategy for data pages.

SCAN Operator (VA = 1)

FROM TABLE

cobis..cl_ente

a

Index : ien_pasapote

Forward Scan.

Positioning by key.

Keys are:

p_pasaporte ASC

Using I/O Size 8 Kbytes for index leaf pages.

With LRU Buffer Replacement Strategy for index leaf pages.

Using I/O Size 8 Kbytes for data pages.

With LRU Buffer Replacement Strategy for data pages.

QP/IO

```
select en_ente, en_nombre, en_subtipo, en_filial, en_oficina, en_ced_ruc, en_fecha_crea
from cobis..cl_ente a
where en_ced_ruc = '0123456789'
union all
select en_ente, en_nombre, en_subtipo, en_filial, en_oficina, en_ced_ruc, en_fecha_crea
from cobis..cl_ente a
where p_pasaporte = '0123456789'
```

Table: cobis..cl_ente (a) scan count 1, logical reads: (regular=4 apf=0 total=4), physical reads: (regular=0 apf=0 total=0), apf IOs used=0

Table: cobis..cl_ente (a) scan count 1, logical reads: (regular=3 apf=0 total=3), physical reads: (regular=0 apf=0 total=0), apf IOs used=0

Total writes for this command: 0



Consideraciones en las condiciones

- ❖ Se debe evitar realizar consultas con el operador **OR**, utilizar union all. Union, utilizarlo solo si se repite los datos en las condiciones.
- ❖ Se debe evaluar el QP cuando se utiliza el operador **IN**. Si se realiza consulta a un subquery que el retorno de datos sean pocos registros (por ejemplo país) Caso contrario realizar un join.
- ❖ Para una lectura legible y rápida en las condiciones, se sugiere que las tablas tengan alias.



Nombre de Procedimientos Almacenados

- ✓ El nombre del procedimiento almacenado debe tener una longitud máxima de 30 caracteres.
- ✓ El nombre del procedimiento almacenado debe empezar con la constante "pa_" + nemónico de la aplicación + "_" + tipo de procedimiento + descripción del procedimiento
- ✓ Los procedimientos almacenados serán desarrollados de acuerdo a la especialidad de la operación, no se permite el uso de opciones. Los tipos de operación permitidos los siguientes:

Tipo	Operación
i	Ingreso
a	Actualización
e	Eliminación
c	Consulta específica
g	Consulta general (Retorno de cientos de registros - rangos)
p	Proceso de cálculo (calcula impuesto)
t	Proceso transaccional (OLTP)
b	Proceso batch - masivo (Batch - select into)
d	Depuración de información

Nombre de Procedimientos Almacenados

Ejemplos:

pa_con_iprovincia	→	Ingreso de provincia
pa_con_aprovincia	→	Actualización de provincia
pa_con_eprovincia	→	Eliminación de provincia
pa_con_bfindemes	→	Batch de fin de mes
pa_con_dmovcontable	→	Depuración de movimientos contables
pa_con_pintmora	→	Proceso de cálculo de interés por mora
pa_con_ccliente_gen	→	Consulta de cliente
pa_con_ccliente_ced	→	Consulta por cliente por cédula
pa_con_ccliente_ruc	→	Consulta por cliente por ruc
pa_con_ccliente_pas	→	Consulta por cliente por pasaporte
pa_con_gcheque	→	Consulta general de cheques



Nombre de Parámetros

- ✓ Si el procedimiento es invocado por la aplicación COBIS, el nombre de los parámetros deben cumplir con el estándar descrito en el Manual de Referencia Técnica de cobis, caso contrario se aplica lo siguiente:
- ✓ El nombre del parámetro debe tener una longitud máxima de 25 caracteres.
- ✓ El nombre del parámetro inicia con el tipo de parámetro + descripción del parámetro.
- ✓ Los tipos de parámetros son:

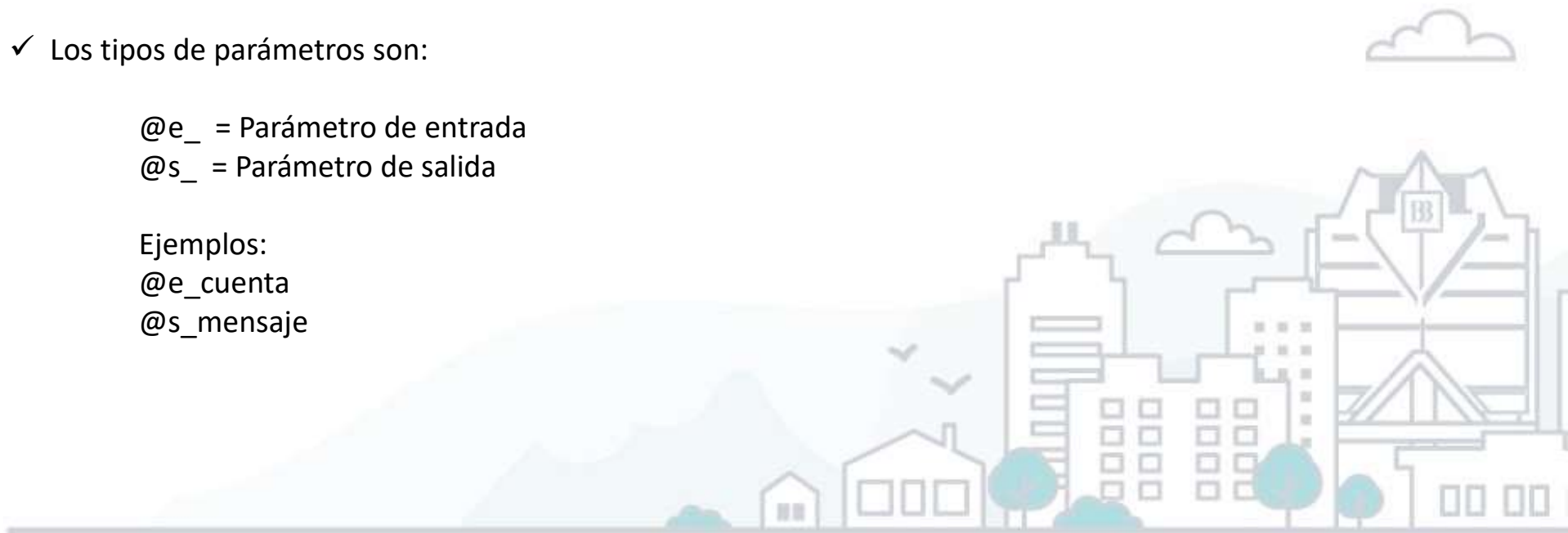
@e_ = Parámetro de entrada

@s_ = Parámetro de salida

Ejemplos:

@e_cuenta

@s_mensaje



Nombre de Funciones de usuario

- ✓ El nombre de la función debe tener una longitud máxima de 30 caracteres.
- ✓ El nombre de la función debe empezar con la constante "fu_" + nemónico de la aplicación + "_" + descripción de la función.
- ✓ No se permite la invocación de funciones en una sentencia DML (Insert/Update/delete/select)

Ejemplo:

fu_con_ultimoDiaMes



Nombre de Variables

Las variables son usadas en scripts, procedimientos y funciones.

- ✓ El nombre de la variable debe tener una longitud máxima de 25 caracteres.
- ✓ La variable se inicia con el carácter @ seguido de la letra "v" + "_" + descripción de variable.

Ejemplo:

@v_ = variable de trabajo



Nombre de Vistas

- ✓ La longitud máxima del nombre de la vista es de 30 caracteres.
- ✓ El nombre de la vista empezará con "vs_" + nemónico de la aplicación + "_" + descripción de la vista.

Ejemplos:

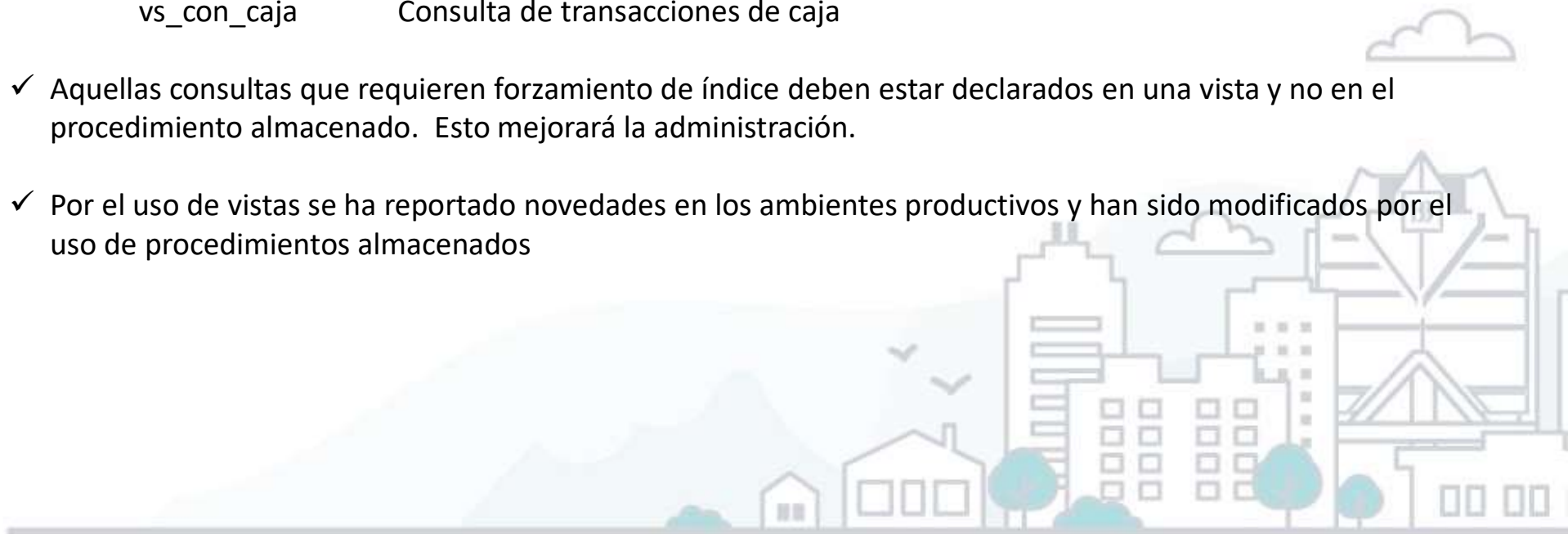
vs_con_abono

Consulta de transacciones de abono

vs_con_caja

Consulta de transacciones de caja

- ✓ Aquellas consultas que requieren forzamiento de índice deben estar declarados en una vista y no en el procedimiento almacenado. Esto mejorará la administración.
- ✓ Por el uso de vistas se ha reportado novedades en los ambientes productivos y han sido modificados por el uso de procedimientos almacenados



Nombre Físico de programas

Nombre físico de procedimientos almacenados/funciones.

Para aplicaciones COBIS

- ✓ Longitud máxima del nombre del programa de 8 caracteres.
- ✓ El nombre debe empezar con el nemónico de la aplicación.
- ✓ El archivo debe tener la extensión sp.

Ejemplos:

ahodepto.sp

conctaco.sp

- ✓ Es obligatorio el comentario de cabecera en los programas .sp cuyo formato sería el que se muestra en el ANEXO 1.

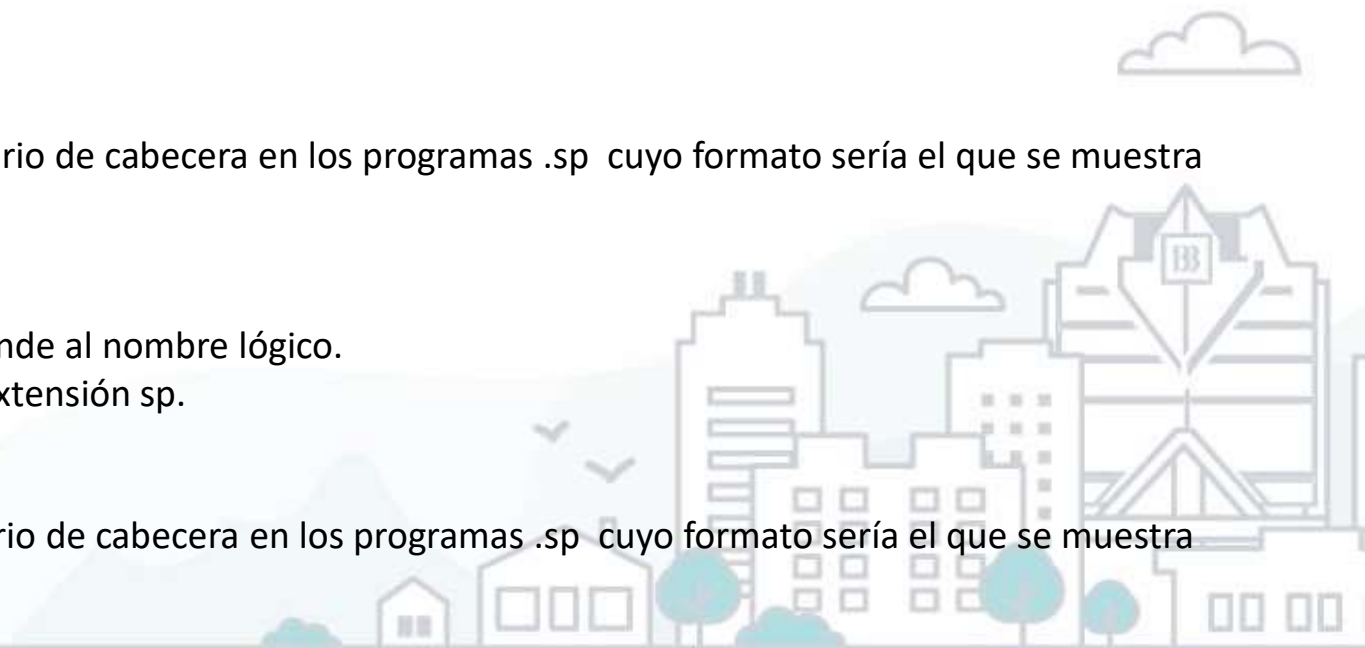
Para aplicaciones NO COBIS

- ✓ El nombre físico corresponde al nombre lógico.
- ✓ El archivo debe tener la extensión sp.

Ejemplos:

pa_int_iempleado.sp

- ✓ Es obligatorio el comentario de cabecera en los programas .sp cuyo formato sería el que se muestra en el ANEXO 1.



Nombre Físico de programas

ANEXO 1

```

/*****
/* Archivo:          bzsp_cobros_e20.sp
/* Motor BD:         sybase
/* Base de datos:    db_biz_admempresa
/* Aplicación:       SAT
/* Stored procedure: pa_c_cobros_cliente
/* Diseñado por:     Julio Cesar Caicedo Bautista
/* Fecha de escritura: 12/Ene/1995
*****/

/*
/* IMPORTANTE
/* "Este programa es parte de los paquetes bancarios propiedad de
/* BANCO BOLIVARIANO S.A.
/* Su uso no autorizado queda expresamente prohibido asi como
/* cualquier alteracion o agregado hecho por alguno de sus
/* usuarios sin el debido consentimiento por escrito de la
/* Presidencia Ejecutiva de BANCO BOLIVARIANO o su representante."
*****/

/* PROPOSITO:
/* OLTP - Obtener informacion de Cobros para E20
*****/

/* MODIFICACIONES:
*****/
/* REF  FECHA      AUTOR          RATIONAL
/* 1  12/Ene/1995 J. Caicedo    SAT-CC-SGC00016690
/*      Emision Inicial
*****/
/* 2  09/Ene/1996 E. Mogro      CTACUM-248-CU-1390
/*      Comentario de fecha de emision por novedad en la consulta
/*      general del cliente
*****/
/* 3  17/Ago/2000 M. Eras        SAT-526-CC-0001
/*      Se adiciona la obtención del nombre del cliente, que se
/*      visualiza en el portal. Se accede a información histórica
*****/
/* 108 27/Mar/2015 O. Velez      SAT-1526-CC-0201
/*      Adición de campo porcentaje de IVA
*****/

```

En el párrafo de PROPOSITO es necesario indicar para que será usado el programa y puede ser OLTP ó BATCH.
El motivo de las modificaciones pueden ser escrito en varias líneas.

Nombre Físico de programas

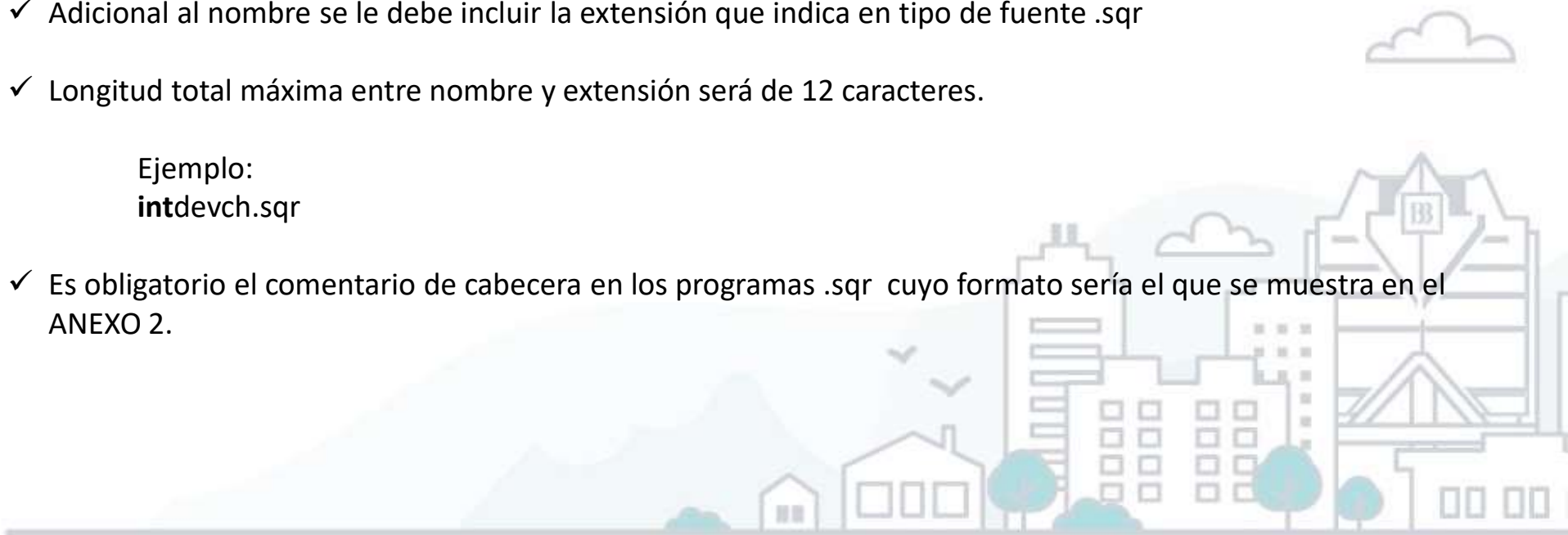
Nombre físico de programas SQR

- ✓ Longitud máxima del nombre del programa será 8 caracteres.
- ✓ El nombre debe empezar con el nemónico de la aplicación.
- ✓ Adicional al nombre se le debe incluir la extensión que indica en tipo de fuente .sqr
- ✓ Longitud total máxima entre nombre y extensión será de 12 caracteres.

Ejemplo:

intdevch.sqr

- ✓ Es obligatorio el comentario de cabecera en los programas .sqr cuyo formato sería el que se muestra en el ANEXO 2.



Nombre Físico de programas

ANEXO 2

```
*****
! * Archivo: ca_fincua.sqr *
! * Base de datos: cob_cartera *
! * Producto: CCA *
! * Diseñado por: Lucia Altamirano B. *
! * Fecha de escritura: 05/NOV/2004 *
! *****
! * IMPORTANTE *
! * Este programa es parte de los paquetes bancarios propiedad de *
! * "BANCO BOLIVARIANO" *
! * Su uso no autorizado queda expresamente prohibido así como *
! * cualquier alteración o agregado hecho por alguno de sus *
! * usuarios sin el debido consentimiento por escrito de la *
! * Presidencia Ejecutiva de BANCO BOLIVARIANO o su representante. *
! *****
! * PROPOSITO *
! * Generar archivo con información de cuadro de información reportada a *
! * procesos de Finanware *
! *****
! * MODIFICACIONES *
! * REF FECHA AUTOR TAREA RACIONAL DESCRIPCION *
! * 1 27/Abr/2010 Adriana Martínez CCA-CC-9914 Migración Sybase 15 *
! * 2 22.jul.2015 Lourdes Reyes CCA-CC-SGC00020808 Cambio x tipoCredito *
! *****
```



Nombre Físico de programas

Nombre de Scripts

- ✓ Longitud máxima del programa de 8 caracteres.
- ✓ El archivo debe tener la extensión .sql.

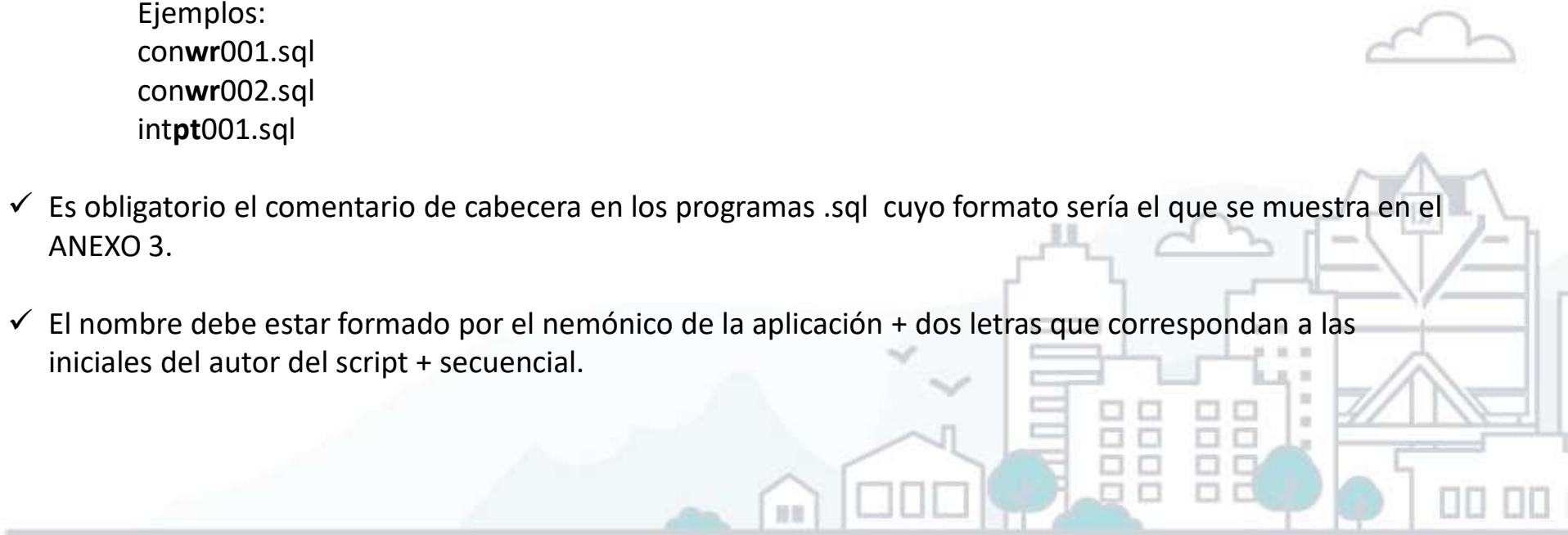
Ejemplos:

con**wr**001.sql

con**wr**002.sql

int**pt**001.sql

- ✓ Es obligatorio el comentario de cabecera en los programas .sql cuyo formato sería el que se muestra en el ANEXO 3.
- ✓ El nombre debe estar formado por el nemónico de la aplicación + dos letras que correspondan a las iniciales del autor del script + secuencial.



Nombre Físico de programas

- ✓ Cuando, en el nombre de los scripts, las 2 iniciales del autor coincidan con las iniciales de otros integrantes del mismo grupo, se deberá alternar entre sus nombres y primer apellido.

Ejemplo:

Winston Vinicio Lazo Ramírez

- WL

- VL

Caso especial

- ✓ Si en las iniciales del autor, sigue existiendo coincidencia, se aplicará la excepción de combinar 3 iniciales entre sus nombres y apellidos, siendo la longitud máxima para este caso de 9 caracteres.

Ejemplo:

Winston Vinicio Lazo Ramírez / Wilmer Vicente López Laz

- WVL

- WLL

- ✓ El nombre debe estar formado por el nemónico de la aplicación + dos letras que correspondan a las iniciales del autor del script + secuencial.

En un script no se permite ejecución de SP, es necesario llevar la ejecución a un SQR

Nombre Físico de programas

ANEXO 3

Nombre script:
Realizado por:
Fecha:
Propósito: (mínimo 2 líneas describiendo el propósito)
Tarea rational:



Nombre Físico de programas

Referencia de cambio en el código adicionado

- Todo cambio al código se marcará el inicio y fin del código de referencia que se ha adicionado. Ejemplo:

/***** MODIFICACIONES *****/			
REF	FECHA	AUTOR	RACIONAL
1	12/Ene/1995	J. Caicedo	SAT-CC-SGC00016690
Emission Inicial			
2	09/Ene/1996	E. Mogro	CTACUM-248-CU-1390
Comentario de fecha de emisión por novedad en la consulta general del cliente			
3	17/Ago/2000	M. Eras	SAT-526-CC-0001
Se adiciona la obtención del nombre del cliente, que se visualiza en el portal. Se accede a información histórica			
108	27/Mar/2015	O. Velez	SAT-1526-CC-0201
Adición de campo porcentaje de IVA			
/*****			
<u>Comentario de campo</u>			
select en_ced_ruc /*<REF 2, fecha_ingreso REF 2>*/			
from cobis..cl_cliente where cl_ente = @i_ente			
<u>Adición de código</u>			
--<REF 3			
select en_ced_ruc, en_nombre			
from cobis..cl_cliente where cl_ente = @i_ente			
--REF 3>			
<u>Comentario de bloque</u>			
/*<REF 4			
select en_ced_ruc, en_nombre			
from cobis..cl_cliente where cl_ente = @i_ente			
REF 4>*/			

GO

- ❖ En Sybase No debe existir TAB antes de la instrucción GO debido a que no se ejecuta correctamente la siguiente sentencia.

```
<TAB>go  
declare c_datos cursor for
```



Control de errores

- ✓ Toda sentencia insert, update, delete debe validar la variable global de error @@error.

```
insert / update / delete / select ... into
if @@error <> 0
begin
    -- Validación de error
    return
end
```

- ✓ En una sola sentencia se debe capturar el código de error y la cantidad de registros afectados y luego realizar la lógica de validación de ambas variables

```
insert / update / delete / select ... into
select @v_error = @@error, @v_afectados = @@rowcount
if @v_error <> 0
begin
    -- Validación de error
    return
end
If @v_afectados = 0
begin
    -- validaciones
end
```



Control de errores

- ✓ En caso de que el procedimiento almacenado sea compilado en un servidor SQLSERVER y es ejecutado desde una aplicación **COBIS** se mantiene el control de error descrito.
- ✓ En caso de que el procedimiento almacenado sea compilado en un servidor SQLSERVER y es ejecutado desde una aplicación **NO COBIS** se sugiere aplicar el siguiente control:

begin try

end try

begin catch

end catch

- ✓ Es necesario que siempre validen el control de error a través de una generación controlada antes de enviar a QA.



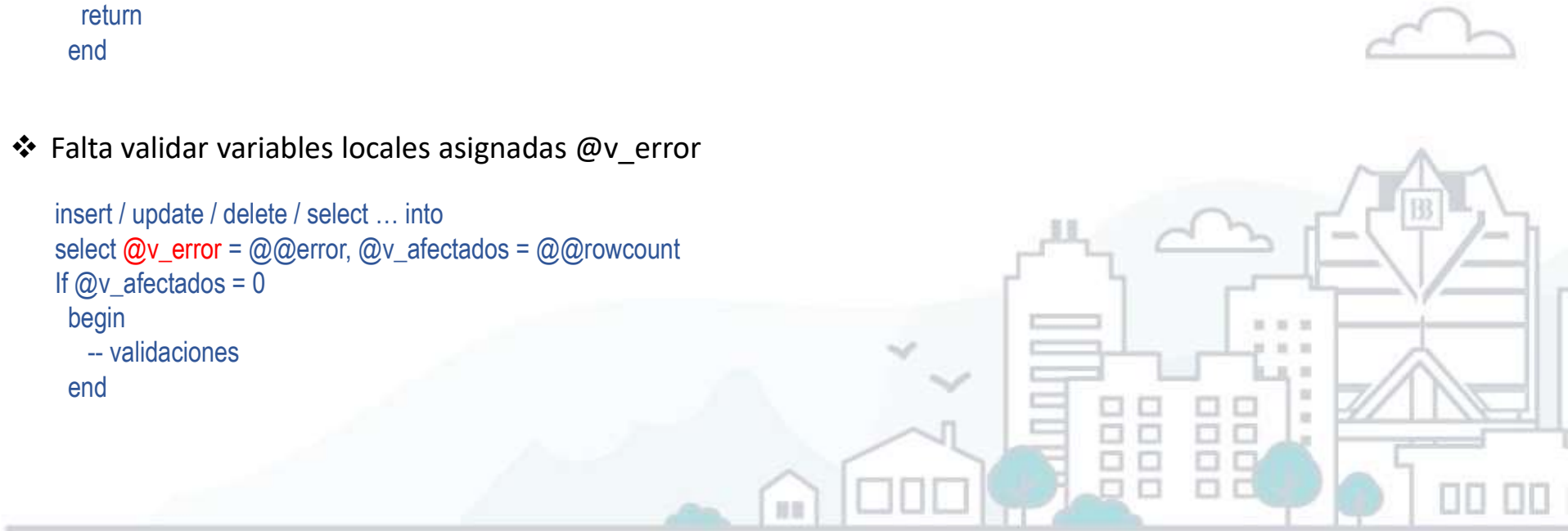
Control de errores – Lógica que se debe evitar

- ❖ No se valida la variable de error previamente capturada sino que se evalúa variable global @@error

```
insert / update / delete / select ... into  
select @v_error = @@error  
if @@error  
begin  
    -- Validación de error  
    return  
end
```

- ❖ Falta validar variables locales asignadas @v_error

```
insert / update / delete / select ... into  
select @v_error = @@error, @v_afectados = @@rowcount  
If @v_afectados = 0  
begin  
    -- validaciones  
end
```



Control de errores – Lógica que se debe evitar

- ❖ Se asigna un valor a la variable local y no es capturado el valor luego de la ejecución de las sentencias.

```
select @v_error = 0
insert / update / delete / select ... into
if @v_error <> 0
begin
    -- Validación de error
    return
end
```

- ❖ Se evalúa la variable global en lugar de la variable local que tiene los valores asignados

```
insert / update / delete / select ... into
select @v_error = @@error, @v_afectados = @@rowcount
If @@error = 0
begin
    -- validaciones
end
If @@rowcount= 0
begin
    -- validaciones
end
```



Control de errores – Lógica que se debe evitar

- ❖ Se debe evitar asignación, impresión de datos antes de liberar recursos de transacción abierta

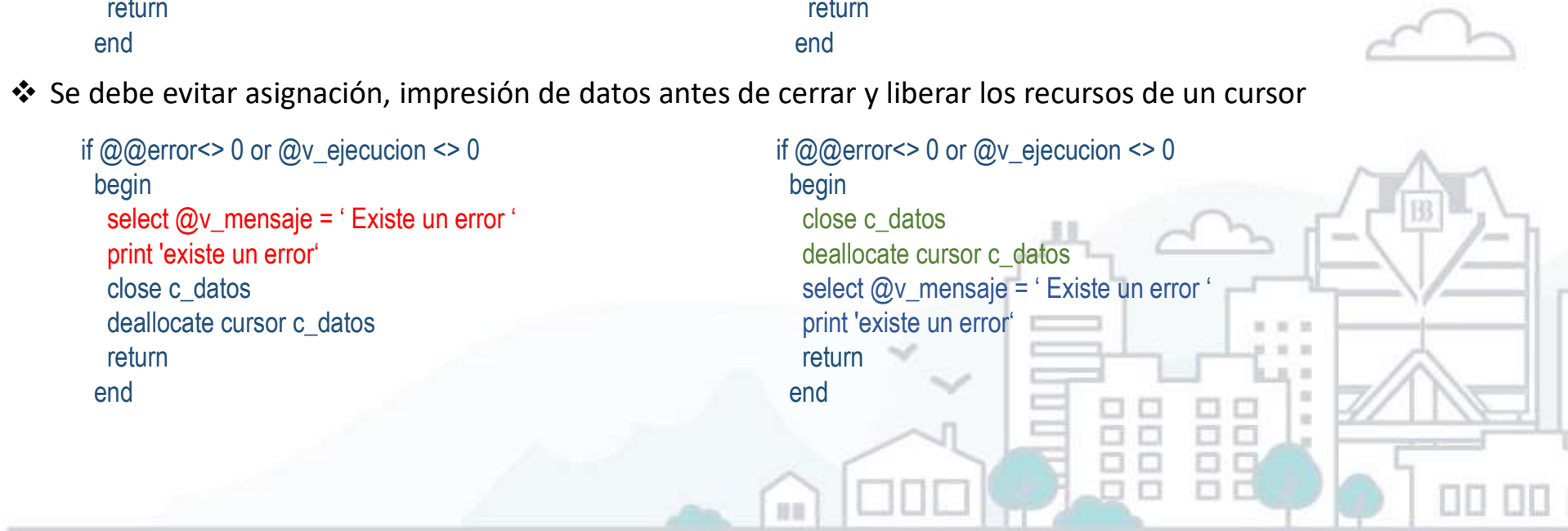
```
if @@error<> 0 or @v_ejecucion <> 0
begin
    select @v_mensaje = ' Existe un error '
    print 'existe un error'
    rollback
    return
end
```

```
if @@error<> 0 or @v_ejecucion <> 0
begin
    rollback
    select @v_mensaje = ' Existe un error '
    print 'existe un error'
    return
end
```

- ❖ Se debe evitar asignación, impresión de datos antes de cerrar y liberar los recursos de un cursor

```
if @@error<> 0 or @v_ejecucion <> 0
begin
    select @v_mensaje = ' Existe un error '
    print 'existe un error'
    close c_datos
    deallocate cursor c_datos
    return
end
```

```
if @@error<> 0 or @v_ejecucion <> 0
begin
    close c_datos
    deallocate cursor c_datos
    select @v_mensaje = ' Existe un error '
    print 'existe un error'
    return
end
```



Control de errores – Lógica que se debe evitar

- ❖ Se debe evitar invocar al procedimiento almacenado de sp_error antes de liberar recursos de transacción abierta.

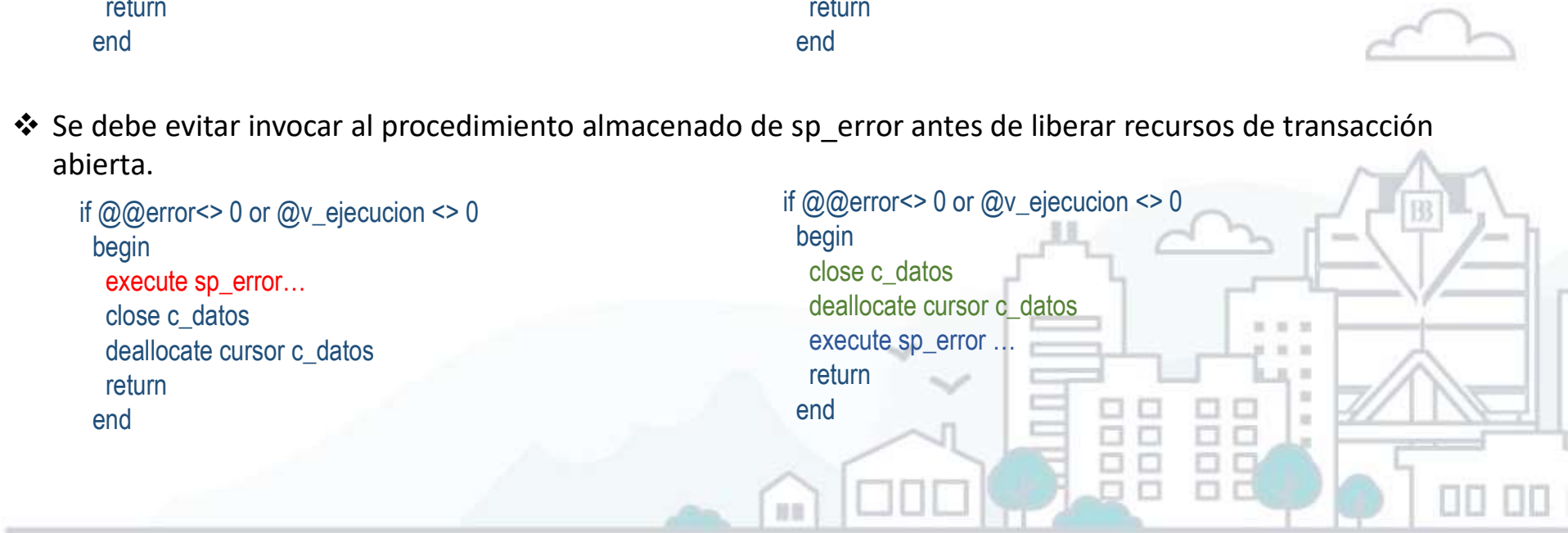
```
if @@error<> 0 or @v_ejecucion <> 0
begin
    execute sp_error ...
    rollback
    return
end
```

```
if @@error<> 0 or @v_ejecucion <> 0
begin
    rollback
    execute sp_error ...
    return
end
```

- ❖ Se debe evitar invocar al procedimiento almacenado de sp_error antes de liberar recursos de transacción abierta.

```
if @@error<> 0 or @v_ejecucion <> 0
begin
    execute sp_error...
    close c_datos
    deallocate cursor c_datos
    return
end
```

```
if @@error<> 0 or @v_ejecucion <> 0
begin
    close c_datos
    deallocate cursor c_datos
    execute sp_error ...
    return
end
```



Control de errores – Lógica que se debe evitar

- ❖ Se debe evitar asignación, impresión de datos antes de invocar al procedimiento de error sp_error

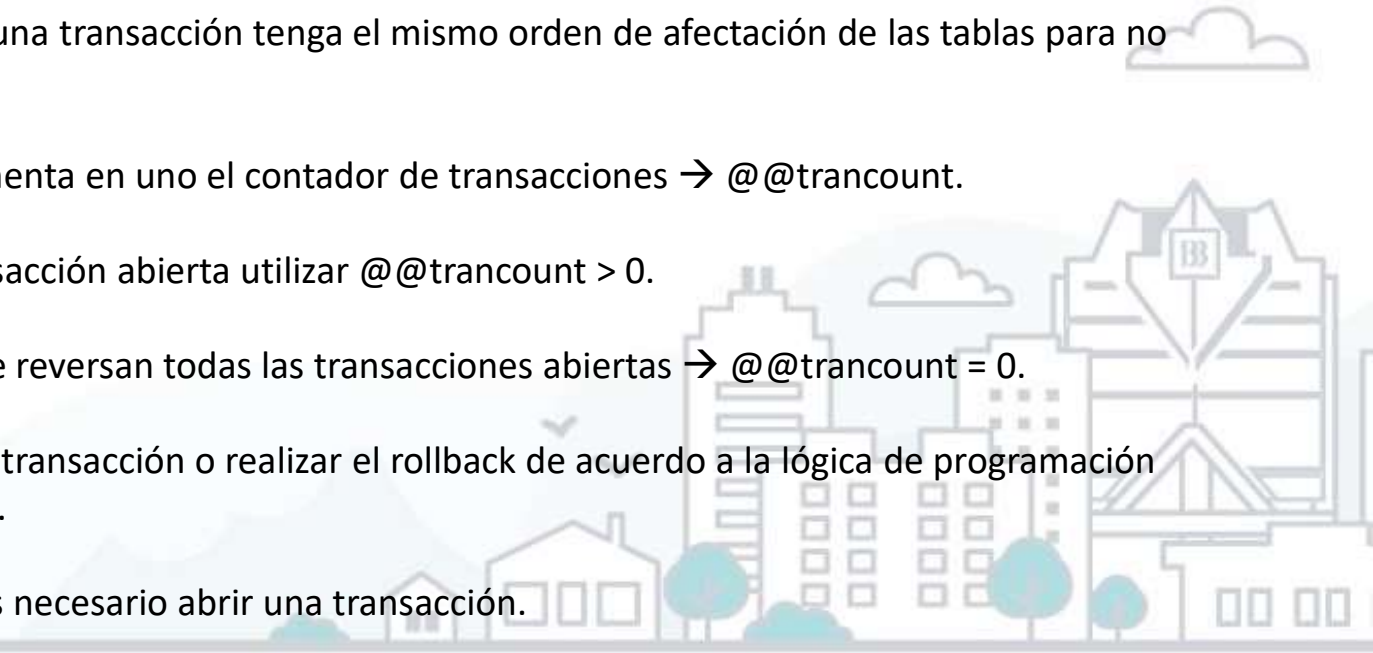
```
if @@error<> 0 or @v_ejecucion <> 0
begin
    select @v_mensaje = ' Existe un error '
    print 'existe un error'
    exec sp_error
    return
end
```

- ❖ Confirmar si después del control de error se continua con la lógica del programa.

```
if @@error<> 0 or @v_ejecucion <> 0
begin
    exec sp_error
end
-- continua procesamiento a pesar de controlar el error
```



Control transaccional

- ✓ Una transacción es usada para que los datos estén consistentes en las tablas.
 - ✓ Siempre debemos pensar que ocurre con la transacción si el motor se indispone.
 - ✓ La transacción debe ser lo más corta posible para evitar bloqueos de otros procesos el cual acceden al mismo recurso.
 - ✓ Es necesario que la lógica de una transacción tenga el mismo orden de afectación de las tablas para no generar un deadlock.
 - ✓ Por cada **begin tran** se incrementa en uno el contador de transacciones → @@trancount.
 - ✓ Para validar si existe una transacción abierta utilizar @@trancount > 0.
 - ✓ Cuando se ejecuta rollback, se reversan todas las transacciones abiertas → @@trancount = 0.
 - ✓ Siempre se debe confirmar la transacción o realizar el rollback de acuerdo a la lógica de programación antes de finalizar el programa.
 - ✓ Para una sentencia DML no es necesario abrir una transacción.
- 

Control transaccional

- ✓ El bus empresarial por defecto inicia una transacción antes de ejecutar un procedimiento almacenado. Es por ese motivo que es **obligatorio** inicie con la siguiente validación:

```
while @@trancount > 0
begin
    commit
end
```

- ✓ Evitar dejar una transacción abierta desde las herramientas de desarrollo – awaiting command.

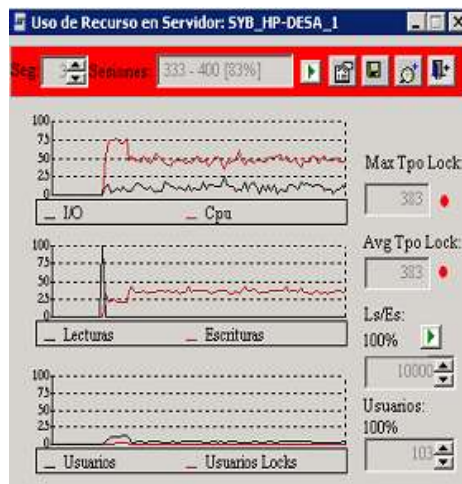
```
-- Simulación de bloqueos del proceso A
select @@trancount
-- 0
begin tran
update qa_m_ente
    set nombre = 'CAMBIO A'
where id_ente = 1
select @@trancount
-- 1
```

```
-- Simulación de bloqueos del proceso B
select @@trancount
-- 0
begin tran
update qa_m_ente
    set nombre = 'CAMBIO B'
where id_ente = 1
-- Espera a que el proceso A termine
```


Control transaccional

- ✓ Monitoreo en tiempo real procesos Sybase a través de Kron Monitor:

spid	Login	Base	Procedimiento	Comando	Linea	IO Físico	CPU	Memoria	Blq	Tiempo Blq	Programa	Fid	Estado
316	mdurann	cobis		UPDATE	2	0	0	12	854	270	microsoft SQL/w		0 lock sleep
910	operator&ct	cobis	SQR. 11420 1633530311	UPDATE	1	0	109	5	0	0	SQR		0 running
941	mdurann	dba control		SELECT	1	0	12	31	0	0	kron.exe		0 running



Plan de Ejecución

PLAN DE EJECUCIÓN

```
update qa_m_ente
set nombre = 'CAMBIO B'
where id_ente = 1
-- Espera a que el proceso A termine
```

Control transaccional

- ✓ Monitoreo en tiempo real procesos Sybase a través de la herramienta interna:

Procesos Sybase DESA01 Fecha: [dd/mm/aaaa] 10/06/2021 Hora: [HH:MM:SS] 11:13:54 AM ☐ Solo Bloqueo

% cpu	% IO	% reads	% writes	#Usuarios	#UsrActivos	#UsrBloqueados	SubMuestra
54	9	0.08	55.17	317	3	1	8076

Spid	Usuario	cpu	Physical_Reads	LogicalReads	Cmd - B L O Q U E O S	LineNumber	RowsAffected	diff_seg
854	mdurann	42	0	0	AWAITING COMMAND	5	0	72216

Spid	Usuario	Comando - S.Q.R.	Tiempo/Min
------	---------	------------------	------------

Spid	Usuario	PhysReads	Comando - ALTO C.P.U.	CPU	LocksHeld	ElapsedTime
------	---------	-----------	-----------------------	-----	-----------	-------------

Spid	Blk	Usuario	PhysReads	LogReads	cpu	Comando - PROCESOS - EXCLUYE LOG/SEG Y SYSMON	#Line	RowsAff	diff_seg
316	854	mdurann	0	0	0	UPDATE	2	0	505
316	0		0	0	0	begin tran	0	0	0
316	0		0	0	0	update qa_m_ente	0	0	0
316	0		0	0	0	set nombre = 'CAMBIO B'	0	0	0
316	0		0	0	0	where id_ente = 1	0	0	0
316	0		0	0	0	-- Espera a que el proceso A termine	0	0	0
782	0		0	0	0	DYNAMIC_SQL [ids000002:	0	0	0
854	0	mdurann	0	0	42	AWAITING COMMAND	5	0	72216
910	0	operadorAct	0	0	178	UPDATE	1	0	6523
910	0		0	0	0	RPC SQR_11420_1633530311;8:	0	0	0
910	0		0	0	6	SQR_11420_1633530311	1	3255	0
941	0		0	0	0	select count (*) from master..sysprocesses where suid > 0	0	0	0

Control transaccional

- ✓ Monitoreo en tiempo real procesos Sql Server a través de la herramienta interna:

Monitor M.A.D.N. - Versión 1.0. Usuario: ADMDBA_MDURANN Wed, 06/10/2021 10:54:06

Monitor Mon/Ora Jobs Disco Trace Log Sybase SqlServer Oracle Distribución TimeOut Desconectado

10:24:03 .02 PROD default/Etapa1.0 oracle.j2ee.ws.client.jaxws.JRFSOAPFaultException: Client received SOAP Fault from server : BEA-380002: connect timed out

10:23:38 .02 PROD default/Etapa1.0 oracle.j2ee.ws.client.jaxws.JRFSOAPFaultException: Client received SOAP Fault from server : BEA-380002: connect timed out

10:23:30 9.28 ESB default/Etapa1.0 oracle.j2ee.ws.client.jaxws.JRFSOAPFaultException: Client received SOAP Fault from server : BEA-380002: connect timed out

Detener Tiempo Real Ver detalle SqlServer Todos Sybase: DESA01 DESA02 HP_ACT HP_REP

Spid	Usuario	Comando	CPU	Physical Reads	Logical Reads	Rows Affected	Elapsed Time	Fec Proceso
1430	apl_osb_hpact	RPC cobis.dbo.pa_CanalesNoCobis_interfase:	651600	43018	0	0	297344000	06/10/2021 10:54:06
300	hpadmsyb	sp_sysmon "00:09:00"	200	6	0	0	244000	06/10/2021 10:54:06

Sybase Central

srv (+ ==> 60sg)	spid	blk	cpu	io	h:m:s	db	host	usr	prg	sql	trx
DESA1SRV	79	--	123	02	00:01:45	cobis	rcadminsrv	admdba_mdurann	Msql Mngt	select @@trancoun	1_awaitin command

SERVIDOR: DESA1SRV

Fecha: [dd/mm/aaaa] 10/06/2021 Hora: [HH:MM:SS] 10:56:05 AM Ir (Fec/Hor Especifica) Buscar Spid por Rango Trx por Hora

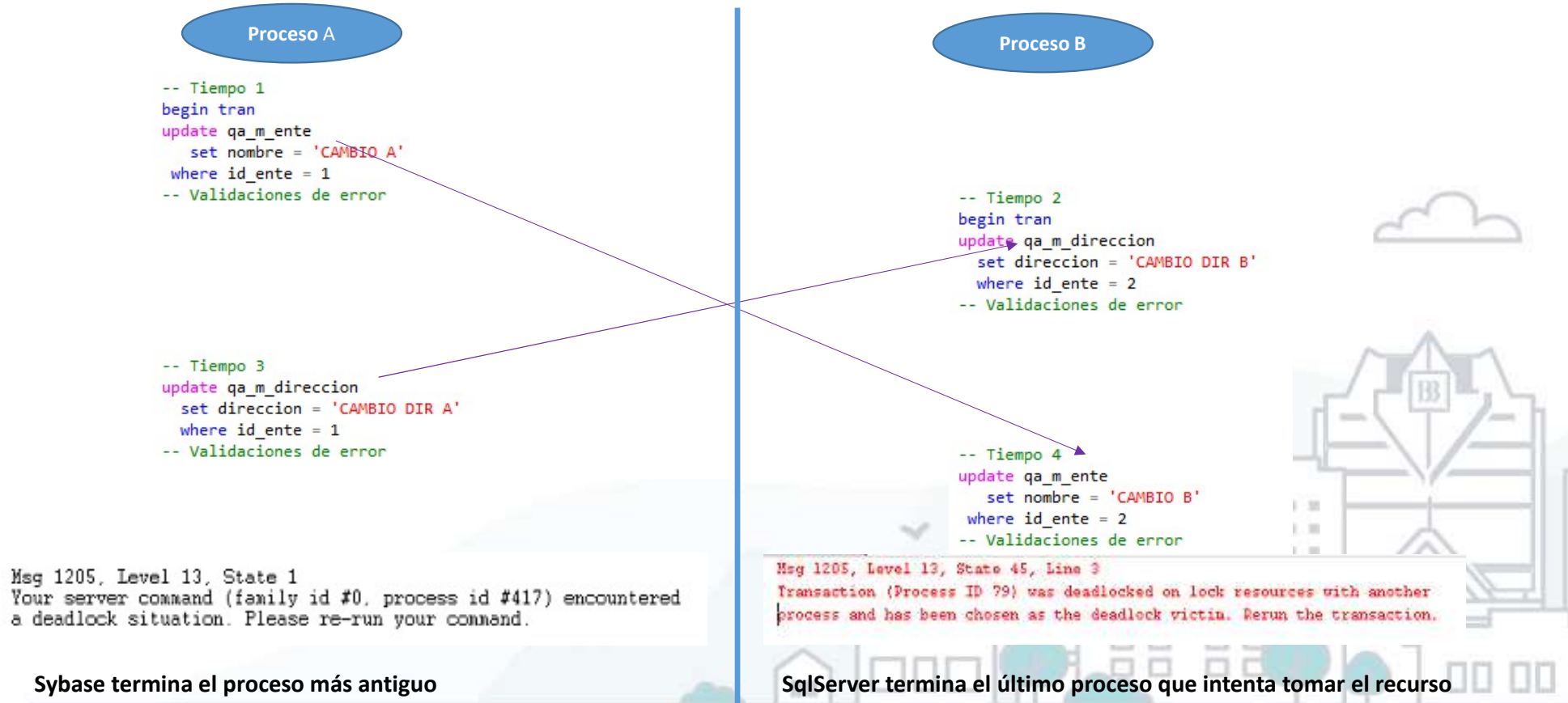
Muestra actual: 182534 Limite: 1 - 182534

Ver detalle Performance <

spid	blk	cpu	io	time	db	host	usr	prg	sql	trx	cmd
79	--	123	62	00:03:45	obis	RCADMINDSRV	MATRIZadmdba_mdurann	Microsoft SQL Server Management Studio - Query	select @@trancoun	1	AWAITING COMMAND
80	79	0	0	00:03:37	obis	RCADMINDSRV	MATRIZadmdba_mdurann	Microsoft SQL Server Management Studio - Query	UPDATE [qa_m_ente] set [nombr	2	UPDATE

DEADLOCK

- ✓ Para evitar el deadlock, abrazo mortal, dentro de una transacción el orden de las tablas debe ser el mismo. Evitar acceso cruzado de las tablas.



DEADLOCK

- ✓ Si el orden de acceso a las tablas es correcto es necesario identificar los índices que acceden la consulta de cada proceso. En este análisis les ayuda en plan de ejecución.
- ✓ Uso de cursores inadecuados. Pueden causar contención en consecuencia poca concurrencia.
- ✓ Si la tabla que genera el deadlock tiene un índice clustered, cambiar por índices non-clustered.
- ✓ Por defecto en Sybase las tablas se crean con el esquema de bloqueo ALLPAGES. **Cada vez que existe un bloqueo a una tabla, se genera un registro de control de bloqueo en la tabla de sistema de Sybase.**
 - Si ha confirmado que el orden de acceso a la tabla es correcto y si es apropiado el acceso del índice, considerar como última opción para evitar el deadlock el cambio de esquema de bloqueo:
 - Cambiar esquema de bloqueo de allpages a datapages, con la instrucción:
alter table table_name lock datapages
Por cada página bloqueada, se genera un registro en la tabla del sistema.
 - Cambiar esquema de bloqueo de datapages a datarows , con la instrucción:
alter table table_name lock datarows
Por cada registro bloqueado, se genera un registro en la tabla del sistema.

DEADLOCK

- ✓ ¡Mantenga las transacciones SQL tan cortas como le sea posible! El mantener bloqueos por largos períodos de tiempo previene que otros usuarios completen sus transacciones.
- ✓ ¡Evite el uso de **holdlock** cuando le sea posible! Esto hace que los bloqueos se mantengan por períodos más largos de tiempo. La única excepción ocurre en situaciones donde se requiere de consistencia de operaciones de lectura dentro de una transacción.
- ✓ Para un mejor rendimiento en bases de datos sometidas a frecuentes operaciones de update, considere diseñar la base de datos para que ocurran operaciones de "actualización en sitio" (**update in place**).
Evitar **Deferred Update**

Las siguientes condiciones se deben cumplir:

- La tabla no puede tener triggers de update.
- Las columnas que se actualicen no pueden ser de longitud variable ni aceptar valores nulos.
- Las columnas que se actualicen no pueden formar parte del índice utilizado en el plan de ejecución del optimizador.
- El update solamente puede afectar una fila a la vez y esta debe poder ser determinada por el optimizador en el plan de consulta (por ejemplo, utilice un argumento de búsqueda que corresponda al índice único).

DEADLOCK

- ✓ Siempre hacer pruebas de carga y tener en cuenta las prácticas remendadas para evitar los deadlock o interbloqueos.



Consideraciones en los procedimientos almacenados

- ✓ Los procedimientos OLTP deben ser especializados en consecuencia tienen poco código.
- ✓ Se deben cerrar inmediatamente las transacciones.
- ✓ En las condiciones los parámetros de entrada deben ser obligatorios y del mismo tipo de datos de los campos que acceden.
- ✓ Se debe evitar la declaración de parámetros por defecto nulo.
- ✓ Evaluar primero sentencias y luego variables

Evitar:

```
if exists(select 1 from <tabla>)  
    and @v_valor > 100
```

Modificar:

```
if @v_valor > 0  
    and exists(select 1...
```

- ✓ Evitar el uso de ciclos infinitos.
while 1=1
- ✓ Se debe evitar la construcción de Query dinámicos– Inyección SQL



Consideraciones en procesos OLTP

- ✓ Se debe evitar el uso de tabla temporal.
- ✓ En caso de necesitar usar una tabla temporal, crear con la instrucción `create table #` para evitar bloqueos en la tabla del sistema.
- ✓ Se permite el uso de la declaración de variable tipo tabla en lugar de utilizar tabla temporal (tempdb). Aunque Sybase soporta esta característica aún esta considerada su uso porque es necesario que el personal de desarrollo realice las pruebas necesarias de uso y de estrés para confirmar si no existe ninguna novedad, Solo está vigente su uso en el motor SqlServer.
- ✓ Cuando se use variables de tipo tabla deben contener pocos registros caso contrario creará una tabla temporal en la base de datos tempdb de manera automática.
- ✓ Si no están dentro de una transacción es posible truncar una tabla con la instrucción **truncate table** y evitar eliminar todos los registros de una tabla con la instrucción `delete`.



Consideraciones de programación

Creación de tablas con columna IDENTITY

- ✓ Se puede crear la tabla con columna identity tanto en el motor Sybase como SqlServer.

```
select id_codigo = identity(int)
...
into cta_tmp_monetaria
from <tabla>
```

Recordar que esta instrucción es más rápida porque no graba en el área de LOG. En Sybase al ejecutar la instrucción puede indisponer a la replicación

- ✓ Sybase:

```
create table cta_tmp_trama(
id int identity,
trama varchar(10)
)
```

- ✓ SqlServer.

```
create table cta_tmp_trama(
id int identity(1, 1),
trama varchar(10)
)
```



Consideraciones en procesos *BATCH*

- ✓ Se permite el uso de tablas temporales.
- ✓ En Sybase se ha observado que el uso de SubQuery ha ocasionado problemas de ejecución por lo cual se considera modificar la lógica a uso de tablas temporales #, teniendo presente el límite 4GB de datos y 2GB de log de la tempdb.
- ✓ Solo en caso de que estemos seguros que se trate de un procesamiento grandes volúmenes de datos se sugiere crear tablas fijas temporales en la base de datos db_temporal.
- ✓ En todo proceso de depuración se sugiere que se realice por bloques de 5.000 registros, de esta manera el consumo del área de LOG de la base datos, es decir, se evita **log suspend**.



Procesos *BATCH*

- ✓ Evitar el uso de Tablas derivadas. En Sybase esporádicamente se caen los procesos. En SqlServer se han quedado sin memoria. Para evitar estas novedades se modifica la lógica a tabla temporal #.
- ✓ Si en un SP, se crea una tabla temporal y requiere procesamiento para acceder por un índice, se debe aplicar el siguiente algoritmo:
 1. Crear tabla temporal # o fija temporal
 2. Poblar con los datos requeridos
 3. Crear índice
 4. Actualizar estadísticas
 5. Invocar al procedimiento interno que accede a la tabla temporal para el optimizador recorra el índice creado



Bases replicadas en Sybase

- ✓ Son 35 bases de datos que se replican en Sybase desde el servidor HP-ACT a HP-REP.
- ✓ Se debe evitar crear tablas físicas con la instrucción select into.
- ✓ En caso de ejecutar esta instrucción los datos no se replican.
- ✓ Si por alguna causa se ejecuta el comando dump transaction <Base> with no_log es necesario volver configurar la replicación.
- ✓ Se debe tener mucho cuidado en crear disparadores - trigger en tablas cuya base se replica. Genera inconsistencia.
- ✓ El comando BCP no graba datos en el área de LOG es por este motivo que al importar datos en SYBASE en una base de datos que se replica no se observa la data replicada en el servidor HP-REP.

cambio
cob_ahorros
cob_atc
cob_atm
cob_cartera
cob_cheques
cob_cobranzas
cob_comext
cob_conta
cob_credito
cob_cuentas
cob_custodia
cob_fondos
cob_giros
cob_internet

cob_leasing
cob_pagos
cob_pfijo
cob_polizas
cob_remesas
cob_rrhh
cob_sbancario
cob_servicios
cob_tesoreria
cob_tvalores
cob_velectron
cob_virtuales
cobis
db_biz_admempresa
db_biz_cobros

db_biz_pagos
db_biz_servicios
db_factoring
db_facturacion
firmas



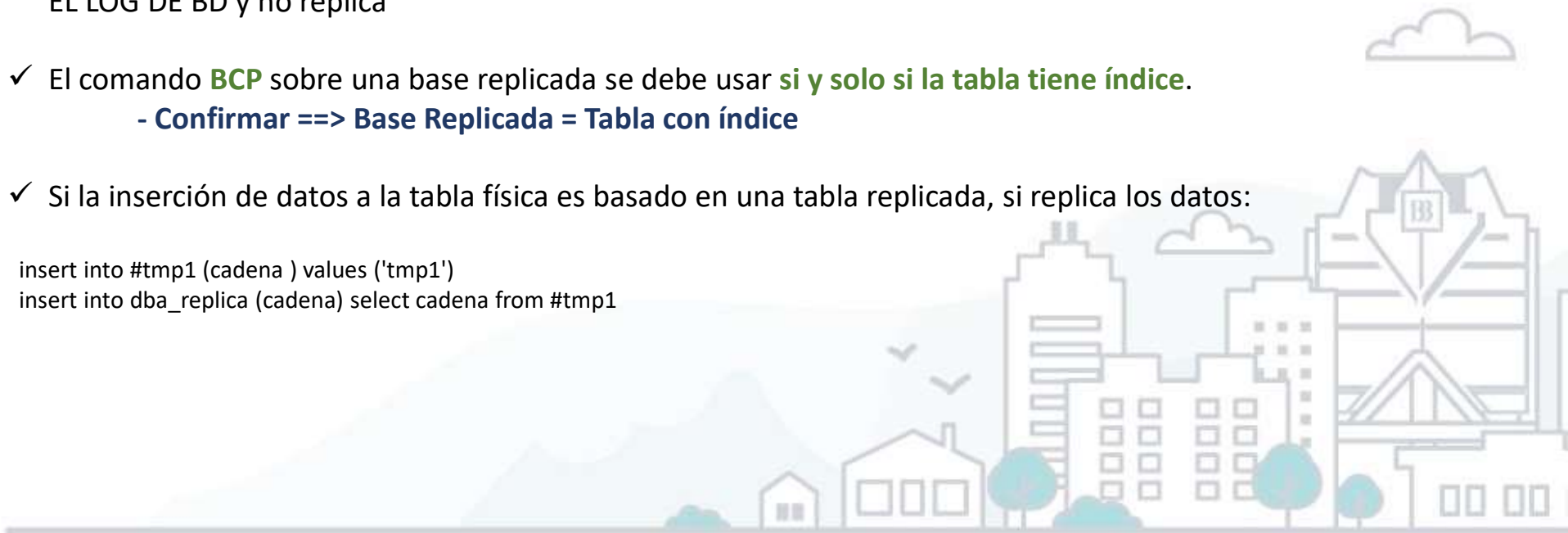
Consideraciones de programación en bases replicadas en SYBASE

- ✓ La instrucción select into para crear tablas físicas en bases de datos que se replican no debe ser usada porque NO GRABA EN EL LOG DE BD y no replica

```
select c1, c2... into dba_replica2 from dba_replica
```

- ✓ El comando BCP sobre base replicada no se debe usar en una tabla SIN INDICE debido a que NO GRABA EN EL LOG DE BD y no replica
- ✓ El comando **BCP** sobre una base replicada se debe usar **si y solo si la tabla tiene índice**.
- **Confirmar ==> Base Replicada = Tabla con índice**
- ✓ Si la inserción de datos a la tabla física es basado en una tabla replicada, si replica los datos:

```
insert into #tmp1 (cadena ) values ('tmp1')  
insert into dba_replica (cadena) select cadena from #tmp1
```



Consideraciones de programación en bases replicadas en SYBASE

- ✓ Una sentencia de actualización basado en una tabla temporal si replica datos:

```
update dba_replica  
set actualiza = 'SI'  
from #tmp1 b  
inner join dba_replica a  
on b.cadena = a.cadena
```

- ✓ Una sentencia de eliminación basado en una tabla temporal si replica datos:

```
delete dba_replica  
from #tmp1 b  
inner join dba_replica a  
on b.cadena = a.cadena
```

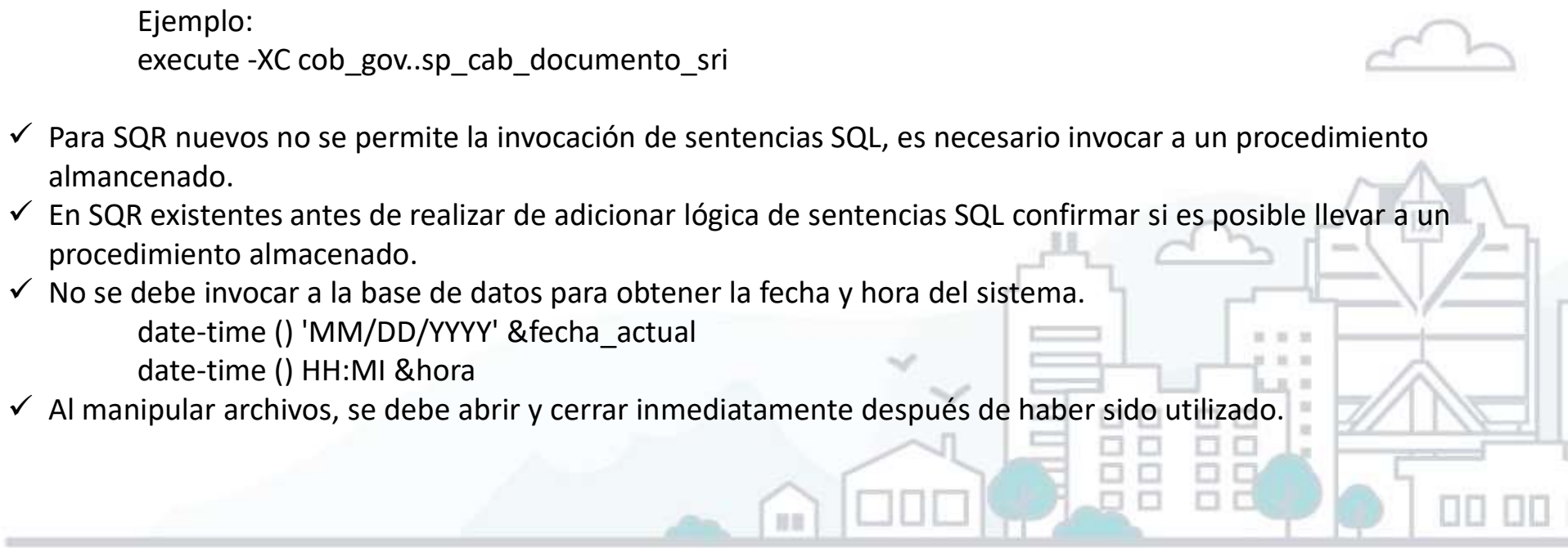


Programas SQR

- ✓ Es permitido el acceso directo a las tablas para aquellos programas SQR cuya funcionalidad interna es la de realizar reportes con quiebres de control, los demás procesos deben invocar a procedimientos almacenados.
- ✓ Todo procedimiento almacenado debe ser invocado con el parámetro –XC para evitar abrir una nueva conexión a la base de datos.

Ejemplo:

```
execute -XC cob_gov..sp_cab_documento_sri
```

- ✓ Para SQR nuevos no se permite la invocación de sentencias SQL, es necesario invocar a un procedimiento almacenado.
 - ✓ En SQR existentes antes de realizar de adicionar lógica de sentencias SQL confirmar si es posible llevar a un procedimiento almacenado.
 - ✓ No se debe invocar a la base de datos para obtener la fecha y hora del sistema.
date-time () 'MM/DD/YYYY' &fecha_actual
date-time () HH:MI &hora
 - ✓ Al manipular archivos, se debe abrir y cerrar inmediatamente después de haber sido utilizado.
- 

Programas SQR

- ✓ Es permitido el acceso directo a las tablas para aquellos programas SQR cuya funcionalidad interna es la de realizar reportes con quiebres de control, los demás procesos deben invocar a procedimientos almacenados.

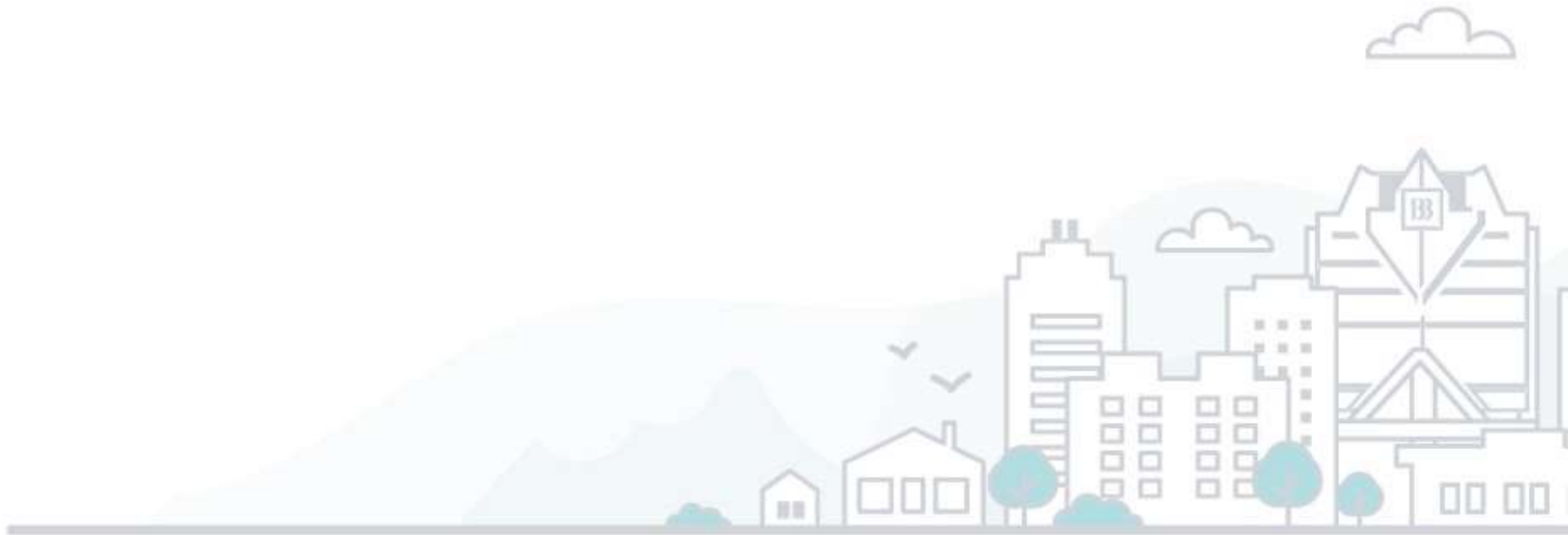
```
-----  
|  
Begin-Procedure Ejecutar_SP  
|  
-----  
execute on-error=Error -XC cob_gov.pa_rot_bcuadre_pasivas  
                        @e_fecha_ini    = $e_fecha_ini  
                        @e_fecha_fin    = $e_fecha_fin  
                        @s_error        = #s_error out  
  
if #s_error <> 0  
  do Error  
end-if  
  
|  
End-Procedure  
|  
-----  
  
|  
Begin-Procedure Error  
|  
  show $sql-error  
  show 'FAVOR COMUNIQUESE URGENTE CON EL STAND BY'  
  do Abortar  
|  
End-Procedure  
|  
-----
```



Algoritmos de apoyo

Proceso de depuración

1. Crear una vista a la tabla cuya condición corresponda a los datos válidos.
2. A través de un SQR extraer los datos a través de un BCP
3. Truncar la tabla
4. Cargar los datos vía BCP por bloques de 5000 registros.



Variables tipo tabla

- ✓ El nombre de la variable debe tener una longitud máxima de 25 caracteres.
- ✓ La variable inicia con el carácter @ seguido de las letras "vt" + "_" + abreviatura de tabla + descripción de variable (campo a usar).

@vt_ = variable de tabla

Ejemplo:

```
define @vt_cl_catalogo_tipocanal table(  
ct_tipo char(1),  
ct_nombre_canal varchar(10)  
)
```

- ✓ Se usada con poca cantidad de registros, si existe muchos registros utilizará tempdb.
- ✓ En un alto nivel de procesamiento generará problemas en la memoria.

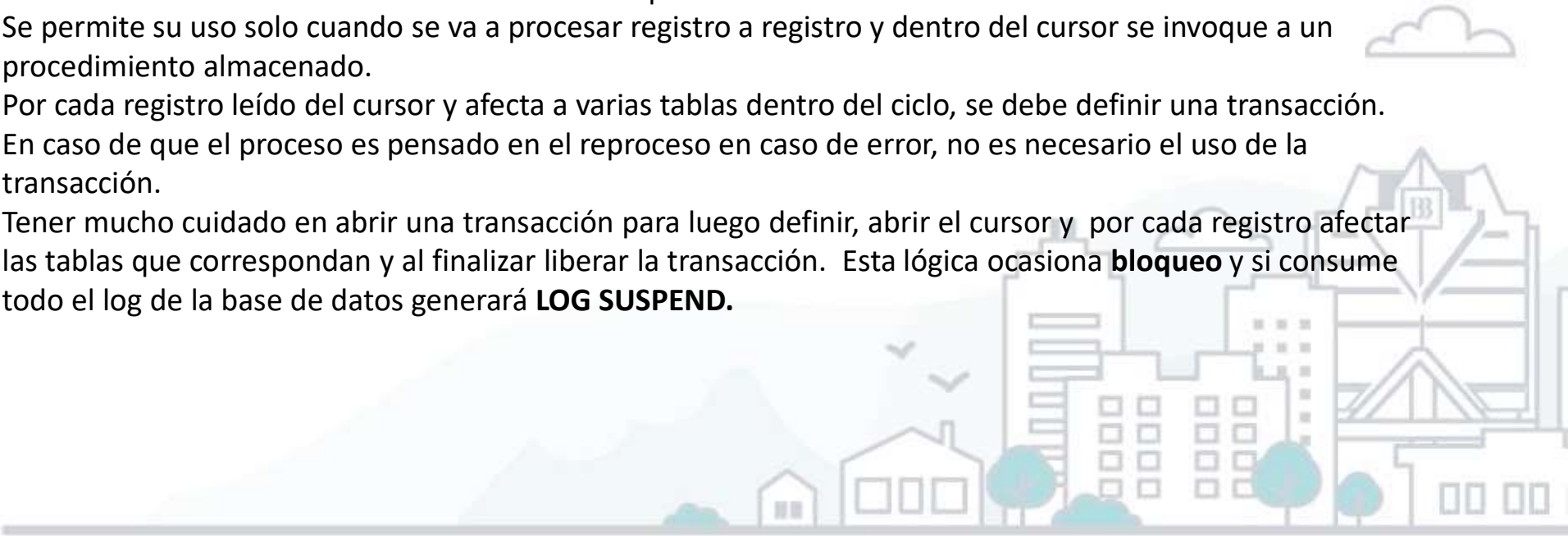


Cursores

- ✓ La longitud máxima del nombre del cursor es de 30 caracteres.
El nombre del cursor empezará con “cu_” + abreviatura tabla + “_” + descripción del cursor.

Ejemplos:

cu_cobinter_clientesbloq → Consulta de clientes bloqueados

- ✓ Recordar que, al finalizar el uso del cursor, éste debe ser cerrado y liberarlo.
 - ✓ Si la sentencia no está correctamente construida puede ocasionar contención.
 - ✓ Se permite su uso solo cuando se va a procesar registro a registro y dentro del cursor se invoque a un procedimiento almacenado.
 - ✓ Por cada registro leído del cursor y afecta a varias tablas dentro del ciclo, se debe definir una transacción.
 - ✓ En caso de que el proceso es pensado en el reproceso en caso de error, no es necesario el uso de la transacción.
 - ✓ Tener mucho cuidado en abrir una transacción para luego definir, abrir el cursor y por cada registro afectar las tablas que correspondan y al finalizar liberar la transacción. Esta lógica ocasiona **bloqueo** y si consume todo el log de la base de datos generará **LOG SUSPEND**.
- 

Uso de cursores artificiales

Consideraciones:

- ✓ Se debe tener mucho cuidado el uso del algoritmo de cursores artificiales porque dependiendo de la lógica puede tomar más tiempo de ejecución.
- ✓ El cursor artificial solo se aplica para disminuir bloqueos a las tablas que intervienen en el proceso pero puede observarse consumo de cpu.



Uso de cursores artificiales

Ejemplo: Se crea una tabla temporal a partir de la cl_ente y se recorre la tabla a través de un cursor artificial.

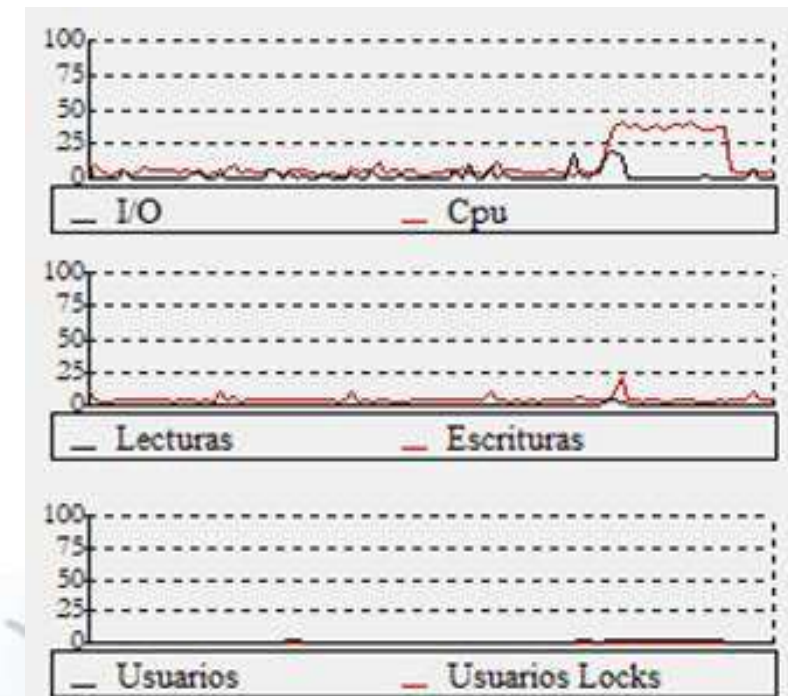
```
select cod_cliente = en_ente
into #sat_tmp_clientes
from cobis..cl_ente
group by en_ente

declare @v_fecha datetime
set @v_fecha = getdate()

create unique clustered index i_sat_tmp_clientes on #sat_tmp_clientes (cod_cliente)
update index statistics #sat_tmp_clientes

select @v_cli_fin = max(cod_cliente)
from #sat_tmp_clientes (index i_sat_tmp_clientes)

select @v_cod_cliente = -1
while @v_cod_cliente < @v_cli_fin
begin
    select top 1 @v_cod_cliente = cod_cliente
    from #sat_tmp_clientes (index i_sat_tmp_clientes)
    where cod_cliente > @v_cod_cliente
    order by cod_cliente
end
```



Cantidad de registros 1'700.000

Tiempo: 49 segundos.

Uso de cursores artificiales

Ejemplo: Se crea una tabla temporal a partir de la cl_ente y se recorre la tabla a través de un cursor.

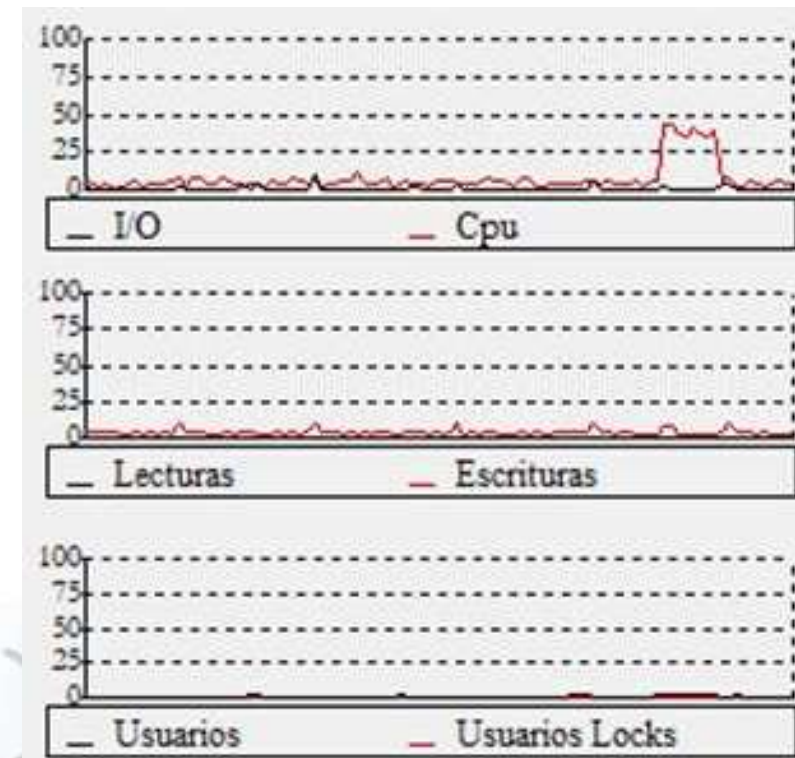
```
select cod_cliente      = en_ente
  into #sat_tmp_clientes
  from cobis..cl_ente
 group by en_ente
-- Validar inserción
declare @v_fecha datetime
set @v_fecha = getdate()

declare c_data cursor for
  select cod_cliente
  from #sat_tmp_clientes

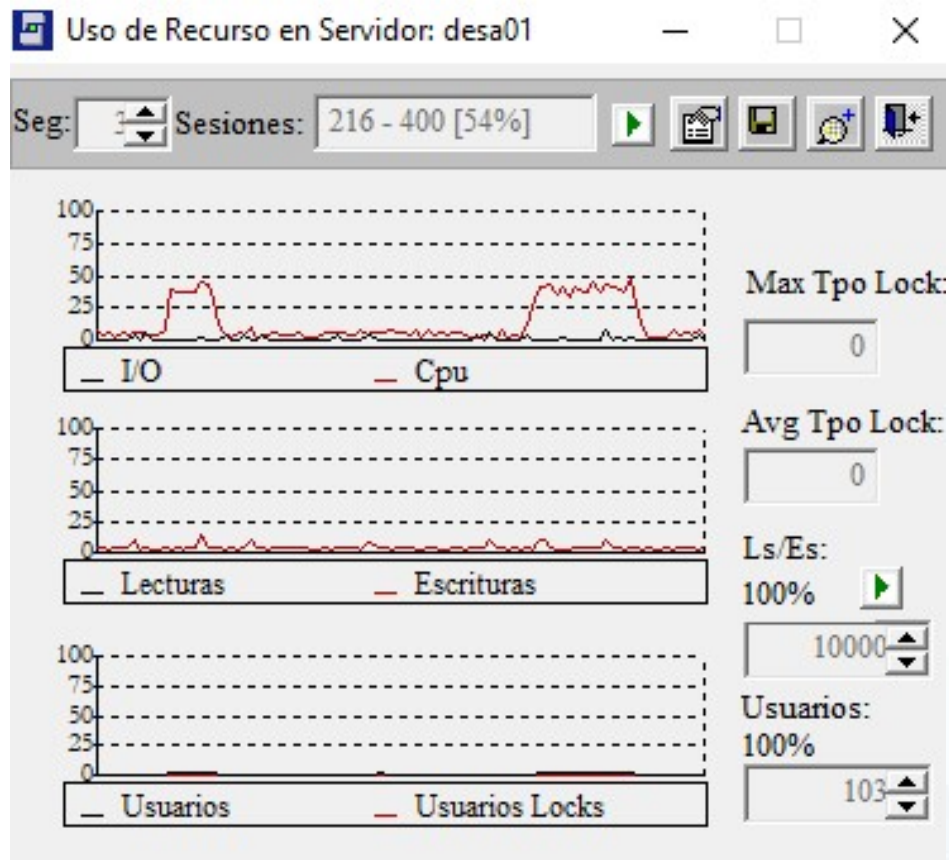
open c_data
fetch c_data into @v_cli_fin
while @@fetch_status = 0
  begin
    fetch c_data into @v_cli_fin
  end
close c_data
deallocate cursor c_data
select datediff(ms, @v_fecha, getdate() )
```

Cantidad de registros 1'700.000

Tiempo: 20 segundos



Cursor vs cursores artificiales accediendo a una tabla temporal



Se observa que el proceso se ejecuta más rápido cuando se accede a la tabla temporal a través de un cursor.

Acerca de las compilaciones en Base de Datos

Consideraciones:

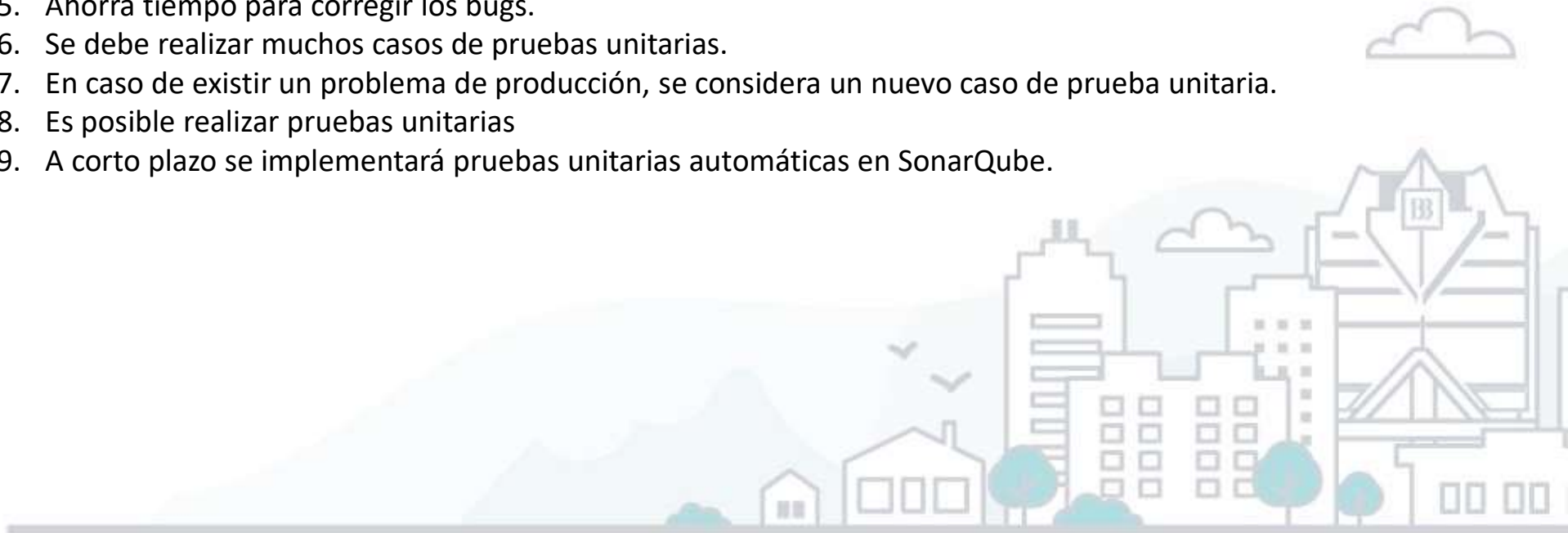
1. Todo script debe ser desarrollado pensando en volver a ser ejecutado. Caso de error de compilación.
2. Todo script de Sybase y SqlServer debe terminar con el comando GO sin espacio ni tab.
3. Todo script de Oracle debe finalizar con el caracter /
4. Las compilaciones de los scripts debe ser realizadas en el ambiente de desarrollo y deben ser ejecutadas:
 - Solo en SqlServer la compilación se realiza a través del Management Studio (SSMS)
 - Para Sybase, la compilación debe ser verificada desde UNIX.
 - Para Oracle, la compilación debe ser verificada desde UNIX y LINUX.



Pruebas Unitarias

Pruebas Unitarias:

1. Verificar lo que se programa.
2. Es posible realizarla con diferentes framework/Lenguajes de programación.
3. El desarrollador adquiere nuevas habilidades.
4. Se evita bug.
5. Ahorra tiempo para corregir los bugs.
6. Se debe realizar muchos casos de pruebas unitarias.
7. En caso de existir un problema de producción, se considera un nuevo caso de prueba unitaria.
8. Es posible realizar pruebas unitarias
9. A corto plazo se implementará pruebas unitarias automáticas en SonarQube.



Pruebas de rendimiento o performance

Prueba de Performance: Investigación Técnica que determina o valida la velocidad, escalabilidad y/o estabilidad de un sistema bajo prueba para analizar su desempeño en situaciones de carga.

- ✓ Comportamiento del tiempo: Tiempo de respuesta que podemos obtener.
- ✓ Utilización de los recursos: Afectación de recursos. Memoria, Disco, etc.
- ✓ Capacidad: Soportar la cantidad de usuarios, Tiempo prolongado de carga, etc.

La prueba de performance, tiene varios tipos de prueba:

Prueba de carga:

- La capacidad que el sistema puede soportar.
- Detectar bugs, memory leaks, procesos zombies – consumen recursos de procesador.

Prueba de estrés:

Carga extrema hasta romperlo.

- ¿Cuál es el cuello de botella?
- Ante la caída de un componente se puede recuperar.
- Quitar los recursos del sistema – Respaldos, procesos pesados en ejecución ¿Cómo se comporta?.



Pruebas de rendimiento o performance

Pruebas de Endurance.

- Pruebas sobre un periodo de tiempo determinado

Pruebas de Escalabilidad

- Proveer una infraestructura para superar la prueba. Adicionar memoria, cpu, etc.

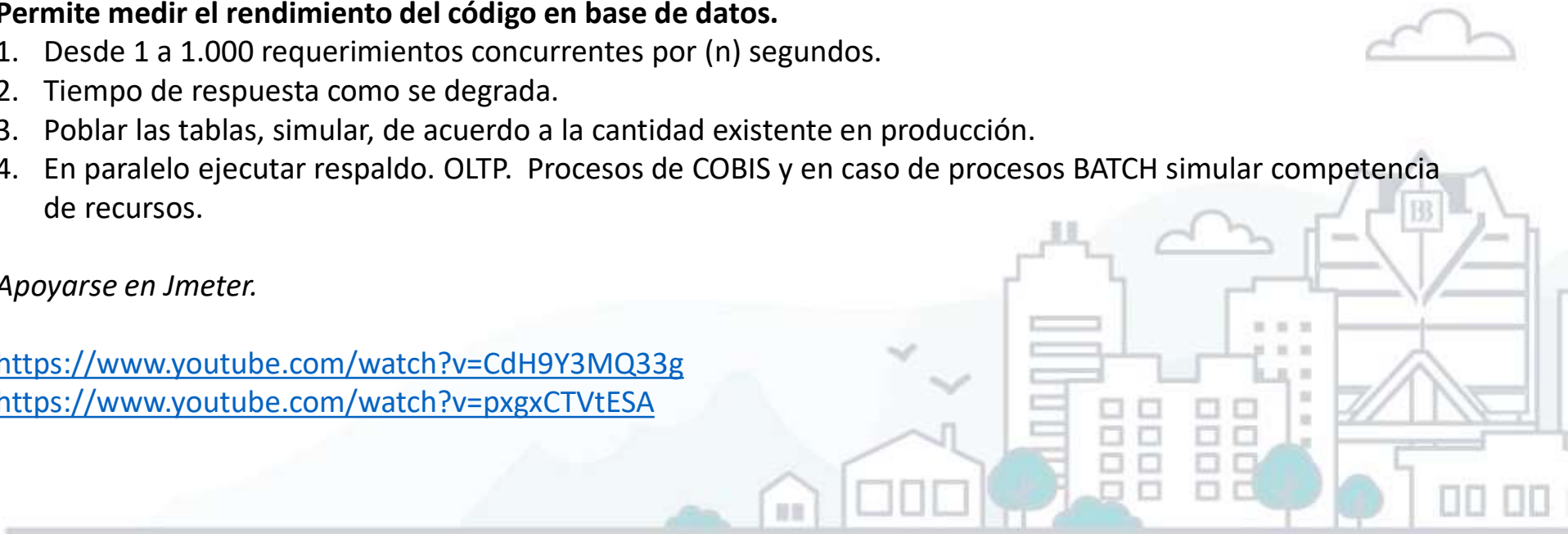
Permite medir el rendimiento del código en base de datos.

1. Desde 1 a 1.000 requerimientos concurrentes por (n) segundos.
2. Tiempo de respuesta como se degrada.
3. Poblar las tablas, simular, de acuerdo a la cantidad existente en producción.
4. En paralelo ejecutar respaldo. OLTP. Procesos de COBIS y en caso de procesos BATCH simular competencia de recursos.

Apoyarse en Jmeter.

<https://www.youtube.com/watch?v=CdH9Y3MQ33g>

<https://www.youtube.com/watch?v=pxgxCTVtESA>



Pruebas de rendimiento o performance



**LOAD
TESTING**



**STRESS
TESTING**

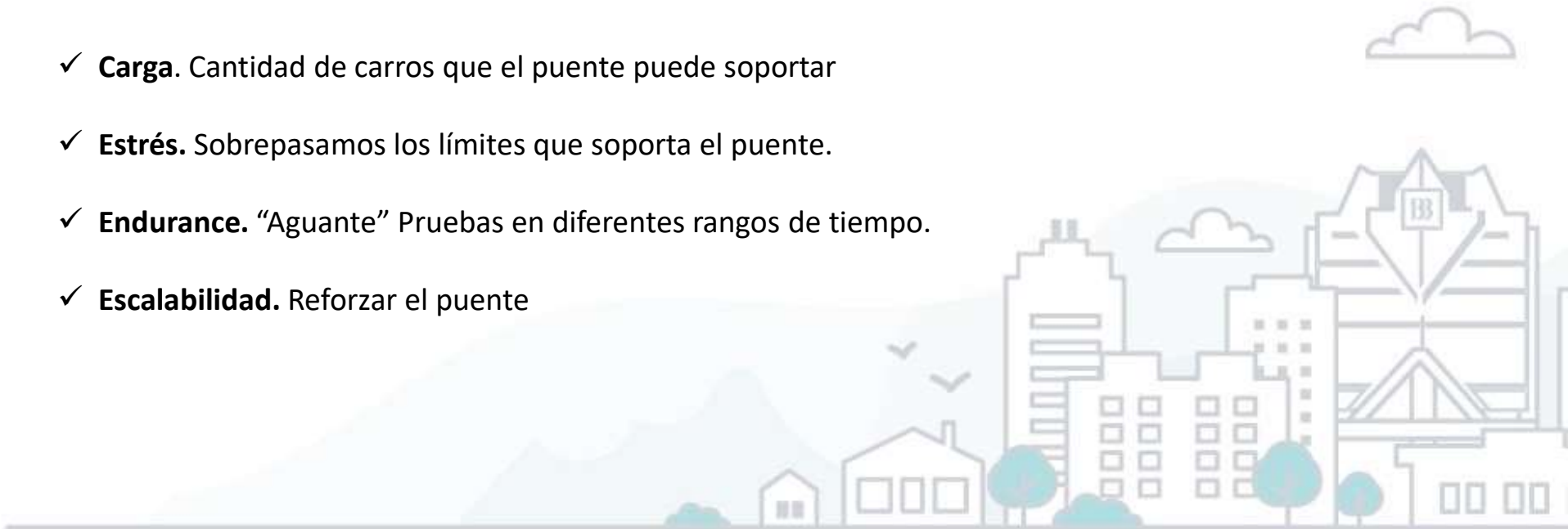


**ENDURANCE
TESTING**



**SCALABILITY
TESTING**

- ✓ **Carga.** Cantidad de carros que el puente puede soportar
- ✓ **Estrés.** Sobrepasamos los límites que soporta el puente.
- ✓ **Endurance.** “Aguante” Pruebas en diferentes rangos de tiempo.
- ✓ **Escalabilidad.** Reforzar el puente



Pruebas de rendimiento o performance

- ✓ Las pruebas demandan recursos, observar lentitud entre procesos.

Número de requerimientos	Tiempo de respuesta (ms)	Tiempo de ejecución de prueba (minuto)	Time Out	Bloqueos	Deadlock	Log Suspend
1	10 ms	1	N	N	N	N
5						
10						
100						
500						



***Desarrollar software cumpliendo los
estándares de calidad, seguridad y de
buen rendimiento es un ARTE.***

M A D N

Gracias

11/Oct/2021

